PUBLIC TECHNICAL DOCUMENTATION

Nexus PM

Software Architecture Document

An AI-powered, cloud-native project management platform built with React 19, FastAPI, PostgreSQL, Cloudflare R2, Redis, Docker, and Google Gemini AI.

Document ID:	NPM-SAD-001
Version:	1.0.0
Classification:	Public
Status:	Production Release
Release Date:	June 26, 2026

Prepared by: Cholan Kinnera

GitHub: github.com/Cholan-kinnera/nexus — **LinkedIn:** linkedin.com/in/cholan-abhaynadh-kinnera01 — **Live Demo:** nexuspm.online

Version 1.0.0 • Released June 2026 • Public Documentation

CHAPTER 1

Revision History

This chapter details the administrative metadata, revision logs, and official sign-off statuses for the Nexus PM Software Architecture Document.

Document Information

Metadata Field	Value
Project Name	Nexus PM
Document Type	Software Architecture Document
Document ID	NPM-SAD-001
Current Version	1.0
Status	Released
Last Updated	June 26, 2026

Revision History

Version	Date	Author	Description
v0.1	June 10, 2026	Cholan Kinnera	Initial document structure and outline
v0.3	June 15, 2026	Cholan Kinnera	Completed requirements and High-Level Design
v0.6	June 18, 2026	Cholan Kinnera	Added Low-Level Design and database schemas
v0.8	June 22, 2026	Cholan Kinnera	Added security structures, deployment layouts, and AI details
v1.0	June 26, 2026	Cholan Kinnera	Initial Public Production Release

Approval

Sign-off Role	Details
Author	Cholan Kinnera
Approver	Principal Software Architect
Status	Released
Distribution	Public

Contents

List of Figures

List of Tables

CHAPTER 2

Abstract

Document Overview

This document presents the formal software architecture of **Nexus PM**, an enterprise-grade, AI-powered project management platform. The purpose of this Software Architecture Document (SAD) is to establish a rigorous, comprehensive blueprint of the system's structural components, design patterns, integration interfaces, and deployment topologies. It serves as a primary technical reference to align development teams, software engineers, systems architects, maintainers, reviewers, and engineering stakeholders on the design decisions that ensure system robustness, scalability, and performance.

Project Scope and Purpose

Nexus PM is designed as a high-performance, full-stack Software-as-a-Service (SaaS) project management platform that redefines modern collaboration workflows. Leveraging advanced generative artificial intelligence, the platform automates complex project management tasks including intelligent task decomposition, resource allocation optimization, automated status reporting, and predictive bottleneck identification. By combining cutting-edge AI features with standard project tracking capabilities (such as Kanban boards, task assignments, and activity feeds), Nexus PM delivers a seamless and highly productive user experience.

Architectural Strategy and Technology Stack

The architectural approach of Nexus PM is rooted in clean separations of concern, scalability, high developer velocity, and robust security. The platform employs a modern decoupled architecture consisting of an optimized frontend client, a high-performance API backend, and cloud-native managed services:

- **Frontend Application Client:** Built with **React 19** and **TypeScript**, optimized for interactive rendering and responsiveness. It utilizes Vite for hot-module reloading and state management via Redux Toolkit and React Query for optimized API state caching. The styling layer is built with TailwindCSS. The frontend is deployed to **Vercel** for global edge delivery.

- **Backend API Engine:** A modular, high-throughput REST API backend implemented in **FastAPI** (Python). It handles request authentication, business logic validation, AI task scheduling, and database access routing. The backend container is orchestrated via **Docker** and deployed on **Render** for automated autoscaling.
- **Data and Cache Infrastructure:** Uses **PostgreSQL** managed by **Supabase** for strong transactional guarantees and relational integrity. **Upstash Redis** acts as a high-speed, distributed in-memory cache for session caching, API rate-limiting tracking, and analytic query aggregation caching.
- **Object Storage:** Powered by S3-compatible **Cloudflare R2** storage to manage user-uploaded document attachments, team avatars, and media assets with low egress costs.
- **Artificial Intelligence Layer:** Integrates state-of-the-art Large Language Models (LLMs) via **Google Gemini 2.5 Flash** to drive natural language task decomposition and automated context summarization.

Intended Audience

This document is structured specifically for:

Software Engineers and Developers as a guide to understanding the code architecture, API design, database schemas, and integration points.

System Architects to review the core patterns, component boundaries, and security paradigms.

Maintainers and Reviewers to evaluate project extensions, coding standards, and contribution guidelines.

DevOps Engineers to review deployment topologies, CI/CD specifications, and system reliability configurations.

Document Roadmap

The remaining chapters of this document systematically detail the Nexus PM design:

- **Chapters 4–6 (Executive Summary, Project Overview, and System Requirements)** establish the strategic goals, technical boundaries, and system constraints.
- **Chapters 7–8 (High-Level and Low-Level Design)** cover the system boundaries, runtime logic flow, component designs, and structural patterns.
- **Chapters 9–10 (Database and API Design)** define the relational database entity structures, query optimizations, and REST API routing schemas.
- **Chapters 11–13 (Security, AI, and Deployment Design)** detail the platform authentication/authorization models, Gemini LLM agent coordination, and containerized deployment schemes.
- **Chapters 14–17 (Testing, Production Readiness, Future Roadmap, and Conclusion)** detail verification guidelines, operational checklists, future technical goals, and summary statements.

CHAPTER 3

Executive Summary

3.1 Introduction

Modern software development and collaboration workflows demand project management tools that are not only highly responsive but also contextually intelligent. Nexus PM is an enterprise-grade, AI-powered project management platform designed to meet these requirements. Architected to deliver the performance characteristics of modern desktop tools, Nexus PM provides a robust collaboration engine that supports real-time state synchronization, secure identity management, role-based access controls, and natively integrated generative artificial intelligence.

This Software Architecture Document (SAD) outlines the comprehensive architectural patterns, cloud-native integrations, and security frameworks that support the Nexus PM ecosystem. The goal of this document is to serve as the definitive blueprint for the system's technical design, detailing how the chosen architecture fulfills enterprise requirements for scalability, availability, data integrity, and operational efficiency.

3.2 Business Problem

Traditional project management applications suffer from several architectural and operational deficiencies:

- **High Cognitive Load:** Project managers and engineers spend a disproportionate amount of time manually breaking down high-level project goals into granular tasks, writing ticket descriptions, and summarizing weekly progress.
- **Performance Bottlenecks:** Legacy platforms built on monolithic web frameworks often experience high latency during page transitions, bulk updates, and real-time synchronization, degrading user experience.
- **Rigid Scaling and High Maintenance Costs:** Monolithic architectures require uniform scaling of the entire application stack, leading to inefficient compute allocation. Furthermore, tight coupling between data layers and business logic makes system updates risky and slows developer velocity.
- **Siloed Data and Isolated Tooling:** Integrating third-party AI tools post-facto often results in synchronization latency, high API costs, and data privacy concerns.

3.3 Proposed Solution

Nexus PM addresses these challenges by introducing a decoupled, cloud-native architecture that combines an ultra-fast, single-page application (SPA) client with a high-performance, asynchronous REST API. The platform features native generative AI capabilities directly embedded into the application lifecycle:

- **Decoupled Execution Model:** The user interface is completely decoupled from the data validation and business logic engine, communicating exclusively through a standardized, secure REST API.
- **Context-Aware Intelligent Automation:** Rather than treating AI as an external add-on, Nexus PM integrates Large Language Models directly into the database and backend flow, enabling automatic task decomposition and automated progress summarization based on live project metadata.
- **Event-Driven and Asynchronous Tasks:** Heavy computations, email notifications, and AI model invocations are handled outside the main request-response cycle, ensuring the user interface remains responsive.

3.4 Technical Highlights

The Nexus PM design leverages industry-standard patterns to maximize maintainability and system longevity:

- **Modular Tiered Backend:** The backend codebase is structured using clean architecture principles, separating models, database repositories, service business logic, and API route controllers. This layer segregation ensures that database engine migrations or external API changes do not leak into core business logic.
- **Strong Relational Modeling:** Database operations are abstracted using the SQLAlchemy Object-Relational Mapper (ORM), enforcing strict validation schemas, relationship cascades, and transactional safety.
- **Client-Side Cache Management:** The client application relies on React Query (TanStack Query) to implement stale-while-revalidate data fetching patterns. This drastically reduces redundant API calls, handles background synchronization, and ensures instant user transitions.

3.5 Cloud Architecture Overview

Nexus PM adopts a cloud-native, managed-service topology designed to minimize operational overhead while guaranteeing high availability:

- **Relational Storage:** Supabase acts as the managed PostgreSQL provider, delivering automated backups, connection pooling, and enterprise-grade row-level security.
- **In-Memory Cache and Rate Limiting:** Upstash Redis provides a fully managed, serverless cache layer. It is used to store user sessions, cache frequently accessed dashboard metrics, and enforce API rate limits at the backend gateway.

- **S3-Compatible Media Storage:** Cloudflare R2 is utilized for object storage of user attachments, team assets, and media files. The choice of R2 eliminates egress fees, reducing recurring cloud storage costs.
- **Transactional Email Pipeline:** Resend is integrated as the dedicated SMTP/API provider, handling automated lifecycle notifications and invite flows.

3.6 AI Capabilities

The core differentiator of Nexus PM is its contextual intelligence engine powered by **Google Gemini 2.5 Flash**:

- **Structured Output Generation:** The backend API integrates with Gemini 2.5 Flash using strict JSON schema constraints (enforced via Pydantic model interfaces). This guarantees that task breakdowns and project summaries returned by the model conform to expected relational structures before being committed to the database.
- **Asynchronous LLM Orchestration:** AI task decomposition and summary jobs are triggered asynchronously, letting the user proceed immediately while background workers handle prompt processing and database insertions.
- **Context Minimization:** Prompts are programmatically built by aggregating only necessary metadata (project descriptions, task titles, status transitions), keeping token consumption low and processing speeds high.

3.7 Scalability and Security

System integrity and horizontal scaling are fundamental pillars of the Nexus PM design:

- **Stateless API Gateways:** The FastAPI backend maintains no local session state, storing all session context in cryptographically signed JWT tokens or Redis cache blocks. This allows backend instances to scale horizontally behind a load balancer without configuration adjustments.
- **Identity and Access Management:** Nexus PM implements dual-layer authentication: secure, stateless local JWT validation alongside Google OAuth identity providers.
- **Role-Based Access Control (RBAC):** A robust access-control matrix distinguishes between Administrators, Project Managers, and Members. Access permissions are evaluated at the API gateway layer using FastAPI dependencies before route handlers execute.

3.8 Deployment Overview

Operational reliability is achieved through automated containerization and cloud hosting platforms:

- **Containerized Environment:** The backend application is packaged inside multi-stage Docker containers. This approach minimizes production image sizes, ensures consistency between development and production environments, and simplifies orchestration.

- **Decoupled Cloud Hosting:** The frontend application is deployed to Vercel, utilizing its global Content Delivery Network (CDN) to serve static resources from edge locations. The backend API is hosted on Render, which manages health-check routing, automated deployments on git push, and automatic scaling groups.

3.9 Key Outcomes

The implementation of the Nexus PM software architecture delivers clear strategic advantages:

- **Sub-Second Response Times:** Through edge-deployed static assets and aggressive Redis caching of API responses, standard user interactions maintain sub-second latencies.
- **Zero-Downtime Deployments:** Blue-green deployments handled via Render and Vercel ensure that software updates do not disrupt active user sessions.
- **High Cost Efficiency:** Leveraging serverless databases, zero-egress object storage (Cloudflare R2), and lightweight backend components allows the platform to sustain high concurrent user counts at a fraction of the cost of traditional monolithic deployments.

3.10 Summary

The Nexus PM Software Architecture Document establishes a clean, decoupled, and cloud-native technical foundation. By separating client interface state from backend database logic, leveraging managed services for storage and caching, and natively embedding asynchronous AI pipelines, Nexus PM successfully balances performance, security, and developer velocity. This architecture is designed to grow with user load, proving fully ready to transition from a development codebase to a resilient, production-ready SaaS enterprise solution.

CHAPTER 4

Architecture at a Glance

4.1 Introduction

This chapter provides a high-level, visual, and quantitative summary of the Nexus PM software architecture. It highlights the technology choices, core architectural principles, cloud deployment targets, and structural metrics that define the platform.

This section serves as a fast-track reference for technical architects, engineering managers, and reviewers before diving into the detailed functional and logical designs of the subsequent chapters.

4.2 System Architecture Overview

Nexus PM is built on a decoupled, cloud-native, and client-centric architecture. The client single-page application (SPA) executes in the user's browser, managing state transitions and interface interactions locally. The backend serves as a stateless API gateway, validating requests, enforcing access control rules, and executing business logic asynchronously where possible to minimize response latency.

The runtime boundary is organized into distinct logical tiers:

- **Presentation Tier:** React 19 single-page application distributed globally via Vercel Edge CDN nodes.
- **Gateway & Logic Tier:** Stateless FastAPI ASGI service running containerized workloads on Render.
- **Persistence Tier:** Managed PostgreSQL (Supabase) handling ACID transaction boundaries, and Cloudflare R2 object storage managing file binary assets.
- **Caching & Orchestration Tier:** Upstash serverless Redis handling transient data (sessions, rate limits, dashboard metrics caching).
- **Intelligence Tier:** Google Gemini 2.5 Flash API integrating natural language processing capabilities.

4.3 Technology Stack Summary

Table 4.1 summarizes the core technology selections, layers, and operational justifications for the Nexus PM ecosystem.

Layer	Technology	Primary Architectural Rationale
Client SPA	React 19, TS, Vite	Fast component-based rendering, static type safety, and fast bundle compilation.
State	Redux Toolkit, React Query	Decoupled client-side UI states and caching for backend queries.
Styling	Tailwind CSS	Rapid interface styling via utility class constraints.
API Service	FastAPI (Python 3.12)	Async-first performance, native OpenAPI docs, and type verification.
ORM	SQLAlchemy, Alembic	Relational mapping, schema version control, and query abstraction.
Database	PostgreSQL (Supabase)	ACID transactional safety, managed database maintenance, and backups.
In-Memory Cache	Upstash Redis	Serverless rate limiting, session caching, and low latency reads.
Object Storage	Cloudflare R2	Zero data egress bandwidth billing, S3-compatible API.
AI Engine	Gemini 2.5 Flash	High context reasoning limits, structured JSON outputs, low cost.
Email Pipeline	Resend API	Developer-friendly transactional email delivery.
Virtualization	Docker	Identical local and production runtime environments.

Table 4.1: Nexus PM Core Technology Selection Summary

4.4 Architectural Principles

The platform's design conforms to four core software engineering paradigms:

- **Complete Client-Server Decoupling:** The frontend client application and backend API service communicate strictly over HTTPS endpoints, enabling independent development, deployment, and scalability.
- **Stateless Gateway Operation:** The API servers store no persistent state on local disk or local memory. All user sessions are validated statelessly using cryptographically signed JSON Web Tokens (JWT) or transient Redis tokens.
- **Non-Blocking Asynchronous Operations:** Heavy computational work, transactional email dispatches, and generative AI model requests are executed asynchronously outside of the primary HTTP request-response cycle.
- **Tenant Resource Isolation:** Multi-tenant isolation is enforced at the API controller layer through database query filters and role validation checks.

4.5 Key Architectural Figures

The detailed structures, interactions, and schemas described throughout this document are represented by six primary diagrams:

- **System Context & Integration Topology:** Figure 7.1 in Chapter 7 (Page 22) shows the operational boundaries and integrations.
- **Runtime Deployment Architecture:** Figure 13.1 in Chapter 13 (Page 70) outlines container network routing.
- **Relational Database Schema (ERD):** Figure 9.1 in Chapter 9 (Page 41) maps tables, primary/foreign keys, and data relationships.
- **Authentication and Identity Verification Flow:** Figure 11.1 in Chapter 11 (Page 59) details the secure sign-in process.
- **Asynchronous AI Execution Pipeline:** Figure 12.1 in Chapter 12 (Page 65) details prompt formatting and structured JSON validations.
- **Zero-Egress Direct Storage Flow:** Figure 8.1 in Chapter 8 (Page 34) shows the pre-signed URL upload lifecycle.

4.6 Key Architecture Statistics

Table 4.2 provides the structural dimensions of the Nexus PM platform as of the current production release.

Architectural Domain	Metric Value	Technical Details
Database Schemas	11 Tables	Mapped via SQLAlchemy ORM models, tracked via Alembic versions.
API Routes	29 Endpoints	Partitioned across 9 routers (Auth, Projects, Members, Tasks, etc.).
Security Protocols	2 Identity Providers	Native credential signup/login and Google OAuth 2.0 federation.
Storage Containers	2 Locations	Supabase PostgreSQL (Structured) and Cloudflare R2 (Unstructured).
Caching Providers	1 Serverless Node	Upstash Redis instance handling sessions and rate-limits.
Testing Level	5 Build Quality Gates	Formatting, Linting, Dependency scan, Migration run, Unit tests.

Table 4.2: Nexus PM Platform Architectural Dimensions

4.7 Chapter Summary

This chapter presented a high-level summary of the Nexus PM architecture, establishing the technologies, principles, deployment structures, and key statistics of the system. The

next chapter will provide the background context of the platform, detailing its business drivers, target personas, system scope, and constraints in the Project Overview.

CHAPTER 5

Project Overview

5.1 Introduction

As software development workflows grow in complexity, the efficiency of collaboration tools directly impacts engineering velocity and product quality. Nexus PM is designed as a modern, full-stack, AI-powered Software-as-a-Service (SaaS) project management platform that bridges the gap between structured project administration and context-aware productivity assistance.

Unlike traditional project tracking tools that act as passive registries of task state, Nexus PM integrates artificial intelligence directly into the collaboration lifecycle, enabling automated requirement analysis, task decomposition, and context-aware project reporting. This chapter provides a high-level overview of Nexus PM, detailing its business vision, core objectives, user personas, system boundaries, technology choices, and architectural constraints.

5.2 Vision

The vision for Nexus PM is to establish a lightweight, highly responsive, and contextually intelligent project management ecosystem. By combining single-page application responsiveness with asynchronous backend services and generative artificial intelligence, Nexus PM aims to reduce the administrative overhead of task management to zero. The platform envisions a collaborative environment where task delegation, technical requirement extraction, and operational updates are automated via secure, cloud-native components.

5.3 Problem Statement

Project teams today face significant friction due to fragmented toolsets and administrative inefficiencies:

- **Administrative Friction:** Writing task descriptions, breaking down large initiatives into tickets, and updating status boards consume valuable engineering and management hours.
- **Context Fragmentation:** Teams frequently jump between task boards, communication applications, file storage systems, and external generative AI tools, leading to

stale task data, mismatched specifications, and security risks.

- **Laggy User Interfaces:** Heavy monolithic tracking systems suffer from slow page reloads and state synchronization delays, interrupting developer workflows.
- **High Infrastructure and Licensing Costs:** Licensing multiple platforms for task tracking, file storage, and AI generation quickly becomes cost-prohibitive for startups, small teams, and individual developers.

5.4 Objectives

The engineering objectives of the Nexus PM architecture are defined as follows:

- **Maximize Client Responsiveness:** Deliver a single-page application (SPA) user interface with sub-second page transitions, optimized resource caching, and instantaneous state updates.
- **Establish Asynchronous Execution Pipelines:** Ensure heavy operations (such as AI task generation, document processing, and email notifications) do not block the primary HTTP request-response cycle.
- **Implement Multi-Tenant Security:** Protect system resources through robust, stateless authorization schemas, role-based access control, and complete database isolation.
- **Optimize Storage and Compute Costs:** Architect a cloud-native platform that leverages serverless components, zero-egress object storage, and lightweight backend services to run efficiently within constrained hosting environments.

5.5 Project Scope

Establishing the system boundaries is critical to defining the operational scope of Nexus PM:

5.5.1 In-Scope Capabilities

The core platform features implemented within the system boundaries include:

- **Identity and Access Management:** Secure user authentication via JSON Web Tokens (JWT) and third-party Google OAuth integration, coupled with granular Role-Based Access Control (RBAC).
- **Project Planning and Tracking:** Interactive Kanban task boards, workspace creation, project categorization, and task assignments.
- **Collaboration Engine:** Comment threads on individual tasks and system-wide activity logging to maintain a clear audit trail.
- **Asset Storage:** Secure file uploads associated with tasks, routed via pre-signed URLs to object storage.
- **Contextual AI Assistance:** Dynamic task breakdown (sub-task generation) and project status summarization leveraging Large Language Models.

- **Notification Pipeline:** Automated email dispatching triggered by project events (e.g., project invites, task assignments).

5.5.2 Out-of-Scope Capabilities

To maintain architectural focus, the following capabilities are explicitly placed outside the system boundaries:

- **Real-Time Video/Audio Chat:** Direct communication is limited to comment threads; real-time calling features are delegated to external communication platforms.
- **Native Git Repository Hosting:** Nexus PM does not host source code. Repository integration is handled via incoming webhooks from platforms like GitHub or GitLab.
- **Billing and Invoicing Engines:** Payment processing is treated as an external modular extension and is not part of the core architectural design.

5.6 Target Users

The Nexus PM platform is optimized for a diverse set of user personas, each with distinct interaction patterns:

- **Individual Developers:** Require a clean, keyboard-navigable task management board with minimal operational overhead.
- **Startups and Small Teams:** Benefit from the AI-assisted requirement generation, which accelerates early-stage product planning without requiring dedicated project managers.
- **Software Engineering Teams:** Rely on robust relational task dependencies, role-based assignments, audit trails, and automated email notifications.
- **Technical Reviewers and Open-Source Maintainers:** Value clean, containerized scaffolding that supports quick local setups and transparent code patterns.
- **Students and Researchers:** Access the platform through cost-free tier configurations to manage educational projects and study workloads.

5.7 Core Features

Architecturally, the features of Nexus PM are grouped into five functional pillars:

1. **Access Control Layer:** Enforces user validation, manages active JWT tokens, and handles Google OAuth redirections. It maps users to specific access permissions (Admin, Project Manager, Member) verified dynamically before route execution.
2. **Relational Planning Engine:** Tracks task status transitions, coordinates task assignments, and manages relationship states between workspaces, projects, tasks, and users.
3. **Asset Management System:** Processes file metadata, validates file formats, and generates secure, short-lived, pre-signed upload URLs, offloading file traffic from the backend API.

4. **AI Orchestration Service:** Collects relevant project context, formats prompt templates, validates structured LLM JSON outputs, and manages backend queues for generative tasks.
5. **Notification Dispatcher:** Evaluates event subscription rules, formats templates, and interfaces with the transactional email gateway.

5.8 Technology Stack Overview

To satisfy performance, scalability, and cost requirements, the technology stack is split into decoupled layers, summarized in Table 5.1.

5.9 System Context

From an architectural boundary perspective, Nexus PM operates within a system context defined by external interfaces and data limits:

- **User Agents:** Web browsers execute the React client application, loading assets via Vercel's global CDN and maintaining secure sessions.
- **API Gateway Boundaries:** All write and read requests go through HTTPS API boundaries managed by FastAPI. The API backend communicates privately with PostgreSQL, Upstash Redis, and third-party REST interfaces.
- **Identity Boundary:** Google OAuth servers handle primary user identity assertion before returning validation codes to the Nexus PM client.
- **Third-Party APIs:** The backend orchestrates calls to Google Gemini AI for content synthesis and Resend for transactional email dispatching, separating external API latencies from the core application threads.

5.10 Constraints

The Nexus PM design is shaped by several critical constraints:

5.10.1 Operational Constraints

- **Free-Tier Cloud Deployment:** The system must run within limited CPU, RAM, and database connection limits provided by Render, Supabase, and Upstash free tiers. This requires strict memory management, database connection pooling, and resource-efficient backend services.
- **Zero-Egress Asset Budget:** Object storage must bypass egress bandwidth fees to ensure operational cost predictability, making Cloudflare R2 the required storage destination.

5.10.2 Architectural Constraints

- **Modular Monolith backend:** To maintain simplicity while planning for future service isolation, the backend is designed as a modular monolith. Modules must communicate via clear interface boundaries, avoiding circular dependencies.
- **Client-First State Management:** To reduce backend processing overhead, the client application is responsible for computing interface state, tracking local state updates, and managing caching rules.

Platform Layer	Technology Choice	Architectural Rationale
Frontend Client	React 19, TypeScript, Vite	High-speed SPA rendering, static type safety, and optimized developer tooling.
State & Caching	Redux Toolkit, React Query	Efficient global UI state management and aggressive server-state caching.
Styling	Tailwind CSS	Rapid UI assembly using a utility-first, highly maintainable CSS structure.
Backend Gateway	FastAPI (Python)	Asynchronous request handling, high throughput, and automatic OpenAPI schema generation.
Database ORM	SQLAlchemy, Alembic	Abstracted data access, migration tracking, and relational data modeling.
Relational Database	PostgreSQL (Supabase)	ACID compliance, transactional guarantees, and real-time database listener capabilities.
In-Memory Cache	Upstash Redis	Serverless key-value storage for API rate limiting, session storage, and rapid page metadata caching.
Object Storage	Cloudflare R2	S3-compatible asset storage featuring zero-egress data transfer fees.
AI Engine	Google Gemini 2.5 Flash	High-speed reasoning, large context window, and support for strict JSON schema outputs.
Email Gateway	Resend	Simple API-driven transactional email dispatching with high deliverability.
Deployment Stack	Docker, Render, Vercel	Containerized backend scaling, CDN-based frontend edge distribution, and automated CI/CD.

Table 5.1: Nexus PM Technology Stack Specification

5.11 Assumptions

The architectural design of Nexus PM relies on the following assumptions:

- **Network Latency:** Users have access to stable internet connections with latencies under 200ms to Vercel and Render edge nodes.
- **SaaS Provider Availability:** Managed infrastructure providers (Supabase, Upstash, Cloudflare, Vercel, Render) maintain at least 99.9% system availability.
- **Security Protocols:** Modern web browsers support HTTPS, secure cookies, and standard JWT parsing mechanisms.

5.12 Chapter Summary

This chapter established the foundational goals and technical scope of the Nexus PM platform. By outlining the target users, core business problems, and architectural objectives, the project context is defined. A modular, decoupled technology stack has been selected to meet the platform's high performance and low-cost deployment constraints. The following chapters will build on this foundation, detailing the system requirements, high-level and low-level software designs, and deployment models.

CHAPTER 6

System Requirements

6.1 Introduction

In software architecture, documenting requirements establishes the baseline against which all design, implementation, and deployment decisions are evaluated. Unlike a business-oriented Software Requirements Specification (SRS), this chapter approaches system requirements from an engineering perspective, detailing the behavioral boundaries (functional requirements) and operational constraints (non-functional requirements) that dictate the structural layout of the Nexus PM platform.

Defining these parameters enables the architecture to balance developer velocity, cost constraints, security policies, and performance characteristics. By mapping system requirements to architectural nodes, the subsequent High-Level Design (HLD) and Low-Level Design (LLD) can directly justify their patterns and schemas.

6.2 Functional Requirements

The functional requirements of Nexus PM are categorized by logical modules to align with the backend's modular structure. Every requirement is defined by a unique identifier, description, priority (High, Medium, Low), and strict verification criteria.

6.2.1 Authentication Module

The authentication module manages user identity, Google OAuth integration, and stateless session lifecycle management.

6.2.2 Projects Module

The projects module manages the workspaces and project metadata structures that group collaborator activities.

ID	Requirement	Priority	Acceptance Criteria
FR-AUTH-001	Google OAuth Integration	High	Users must be able to authenticate using Google OAuth 2.0. The backend must exchange authorization codes for identity tokens and match or provision accounts.
FR-AUTH-002	JWT Session Management	High	Successful authentication must issue a cryptographically signed access token (short-lived) and refresh token (long-lived) stored in secure, HttpOnly, SameSite cookies.
FR-AUTH-003	Session Revocation	High	Initiating a logout request must invalidate the session token on the client and purge the corresponding refresh token token block from the cache.

Table 6.1: Authentication Functional Requirements

ID	Requirement	Priority	Acceptance Criteria
FR-PROJ-001	Workspace Isolation	High	All data (projects, tasks, members) must belong to a parent Workspace. Users cannot query, view, or modify resources belonging to workspaces where they are not active members.
FR-PROJ-002	Project Configuration	High	Members with appropriate roles must be able to create, update, and soft-delete projects within a workspace, specifying name, description, and status attributes.

Table 6.2: Projects Functional Requirements

6.2.3 Tasks Module

The tasks module manages the interactive planning canvas, task fields, and Kanban lanes.

ID	Requirement	Priority	Acceptance Criteria
FR-TASK-001	Kanban Board State	High	Task entities must map to status lanes (e.g., Todo, In Progress, Review, Done). Drag-and-drop actions on the client must trigger an immediate backend update, returning the updated task sequence.
FR-TASK-002	Assignments & Links	Medium	Tasks must support assignment to valid workspace users and support task-to-task relationships (e.g., blocked by, duplicates, blocks).
FR-TASK-003	Collaboration Logs	Medium	Users must be able to add comments to tasks. Any change in task state (status, assignee, priority) must append a record to the workspace activity log.

Table 6.3: Tasks Functional Requirements

6.2.4 Storage Module

The storage module interfaces with object storage to support project attachments without bloating the primary database.

ID	Requirement	Priority	Acceptance Criteria
FR-STOR-001	Pre-signed Uploads	High	The client must request a secure pre-signed URL from the backend before uploading file attachments. The actual file upload must bypass the backend API and upload directly to Cloudflare R2.
FR-STOR-002	Attachment Linking	High	Upon successful upload, file metadata (key, content-type, file size) must be sent to the API to link the file record to the corresponding task entity.

Table 6.4: Storage Functional Requirements

6.2.5 Notifications Module

The notifications module dispatches system alerts to ensure team alignment.

ID	Requirement	Priority	Acceptance Criteria
FR-NOTI-001	Invitation Emails	High	Inviting users to a workspace must trigger a secure sign-up link dispatched via email containing token signatures.
FR-NOTI-002	Event Notifications	Medium	Task assignments and status changes must generate in-app alerts and trigger background email notifications to affected workspace collaborators.

Table 6.5: Notifications Functional Requirements

6.2.6 Analytics Module

The analytics module computes data representations of project state and velocity.

ID	Requirement	Priority	Acceptance Criteria
FR-ANLY-001	Workspace Dashboard	Medium	The dashboard must calculate active task counts, completion velocities, and workload distributions across workspace members for a selected time window.

Table 6.6: Analytics Functional Requirements

6.2.7 AI Module

The AI module coordinates LLM interactions to augment standard task management tasks.

ID	Requirement	Priority	Acceptance Criteria
FR-AI-001	Task Decomposition	Medium	Given a high-level task title and description, the AI service must generate structured sub-tasks containing titles, estimated durations, and priority levels.
FR-AI-002	Project Summarization	Medium	The backend must aggregate active task descriptions and recent change logs to produce a structured natural-language status report.

Table 6.7: AI Functional Requirements

6.2.8 Administration Module

The administration module provides access control overrides and logs verification.

ID	Requirement	Priority	Acceptance Criteria
FR-ADMIN-001	Workspace RBAC	High	Workspace administrators must be able to change roles of users within their workspace, verifying access control permissions dynamically.
FR-ADMIN-002	Audit Log Inspection	Medium	The system must record write operations (inserts, updates, deletes) in an immutable log table containing timestamps, actor identities, and entity changes.

Table 6.8: Administration Functional Requirements

6.3 Non-Functional Requirements

Non-functional requirements (NFRs) specify the operational characteristics, constraints, and quality attributes of the system. These parameters shape the architectural patterns selected for the platform.

- **Performance:** Maximum API response times for read paths must remain below 100ms under standard load conditions. Client-side state transitions (such as switching views on the Kanban board) must resolve within 50ms. High client responsiveness is critical to match desktop application experiences.
- **Scalability:** The API backend must be stateless to support horizontal scaling across auto-scaling compute groups. Database access must utilize connection pooling to prevent connection exhaust issues during transaction spikes.
- **Availability:** The core platform target availability is 99.9% during regular operations, excluding scheduled maintenance. This requires decoupled integrations where the failure of secondary services (such as email delivery or AI summaries) does not prevent users from accessing the task database.
- **Reliability:** Relational data must maintain strict ACID transactions. Relational integrity must be enforced at the database level, with automated daily backups provided by the managed database host.
- **Maintainability:** The backend codebase must enforce strict layered boundaries (Repository $\rightarrow$ Service $\rightarrow$ Router) to allow independent database, business logic, and API modifications.
- **Security:** All endpoints must enforce transport layer security (TLS 1.3). Access tokens must be verified statelessly, and user passwords must be hashed using bcrypt before database storage.
- **Usability:** The user interface must present desktop-like transitions, keyboard shortcuts, and responsive layouts that scale gracefully from mobile viewports to large workspace displays.
- **Accessibility:** Frontend templates must utilize semantic HTML tags, support keyboard-only navigation, and satisfy core contrast ratios to comply with accessibility best practices.
- **Compatibility:** The single-page application client must be verified to operate correctly across all modern web browsers, including Google Chrome, Apple Safari, Mozilla Firefox, and Microsoft Edge.
- **Disaster Recovery:** The platform must configure automated, geographical-redundant database backups with a target Recovery Point Objective (RPO) under 24 hours and a Recovery Time Objective (RTO) under 4 hours.
- **Cloud Deployment:** The system must build inside declarative, multi-stage Docker images to guarantee deployment parity between development, testing, staging, and production environments.

6.4 External System Requirements

To minimize operational overhead and focus core application development on project management workflows, Nexus PM integrates with external cloud-native services. The role and integration specifications of each service are detailed below:

- **Supabase (PostgreSQL):** Acts as the primary transactional database. The integration requires secure PostgreSQL connections over TLS, connection routing via PgBouncer pooling layers, and row-level security (RLS) configurations.
- **Cloudflare R2:** Provides S3-compatible, zero-egress object storage. The backend interfaces with R2 using AWS SDK clients to generate temporary pre-signed upload URLs and access-controlled download tokens.
- **Upstash Redis:** Serves as the serverless in-memory cache and session registry. It handles rate-limiting tracking and transient metadata storage, communicating via secure Redis database connection protocols.
- **Resend:** Serves as the transactional email dispatch engine. The backend routes notifications to Resend via their HTTPS API, passing structured JSON payloads containing HTML mail templates.
- **Google OAuth Server:** Acts as the third-party identity validation provider, using standard OAuth 2.0 authorization code exchange endpoints to verify user credentials.
- **Google Gemini AI (API Gateway):** Provides the LLM generation capabilities. The backend interacts with Google's API gateway asynchronously, using structured schema parameters to enforce rigid JSON response types.

6.5 Security Requirements

The security model of Nexus PM is designed to protect sensitive workspace data and prevent unauthorized system access:

- **Stateless Access Control:** User authentication is validated via JSON Web Tokens (JWT) signed with HMAC-SHA256 using server-side keys. Access tokens are short-lived (e.g., 15 minutes) and must be paired with database-backed refresh tokens for session renewal.
- **Cookie Containment Security:** All authentication tokens must reside in cookies flagged as `HttpOnly` (preventing client-side JavaScript access), `Secure` (enforcing transmission over HTTPS only), and `SameSite=Strict` (mitigating Cross-Site Request Forgery risks).
- **Input Validation and Sanitization:** Every incoming request payload must be parsed, validated, and sanitized against strict Pydantic schemas before reaching backend controller functions.
- **Role-Based Permissions (RBAC):** Endpoints are protected by a declarative permission matrix. The gateway dynamically evaluates active user roles (Admin, Manager, Member) using FastAPI dependency injection structures.

- **File Security Validation:** Pre-signed URL requests must specify file sizes and MIME types, which are validated against whitelist restrictions to block malicious file executions on client browsers.
- **API Gateway Rate Limiting:** To protect API resources, rate-limits are tracked in Upstash Redis and evaluated per IP address and authenticated user ID.

6.6 Performance Targets

The target runtime metrics for Nexus PM are defined as follows:

- **API Latency (Standard Operations):** Read-heavy HTTP requests (e.g., fetching task lists or workspace dashboards) must complete within 100ms. Write-heavy operations (e.g., creating tasks, appending comments) must complete within 200ms.
- **Database Query Execution:** The database engine must execute indexed queries in under 20ms under baseline loads.
- **File Upload Response:** The API must generate pre-signed upload URLs within 50ms, allowing direct uploads to object storage without backend overhead.
- **Cold Start Mitigation:** Free-tier container hosting limitations (e.g., Render service sleep modes) must be managed using warm-up ping engines, aiming to resolve initial request times under 30 seconds for dormant instances.
- **Cache Performance:** The system must maintain a Redis cache hit ratio greater than 85% for frequently accessed, read-only dashboard calculations.

6.7 Constraints

The technical design of Nexus PM is bounded by the following real-world constraints:

- **Free-Tier Infrastructure Limits:** Web service hosting (Render) restricts container instances to a single CPU core and 512MB of RAM, causing cold start delays. Relational database limits restrict active connections and database size, making efficient resource use critical.
- **API Limit Restrictions:** External APIs (Google Gemini, Resend) enforce maximum monthly request quotas and rate limits, requiring local request caching and back-off handling.
- **Development Budget Constraints:** Zero-budget configurations prevent the use of paid database clusters or dedicated egress networks, requiring architectures optimized for zero-cost managed services.
- **Client Compatibility:** The web application client must support standard HTML5, CSS3, and ES6 Javascript versions, avoiding dependencies on proprietary client extensions.

6.8 Assumptions

The architectural blueprints for the platform rely on the following assertions:

- **Network Integrity:** Network latency between Vercel edge routes, Render compute instances, and Supabase database endpoints remains under 50ms.
- **Cloud Service Availability:** Managed cloud services maintain at least 99.9% availability during runtime.
- **Browser Security:** User clients respect security tags such as `HttpOnly` cookies, CORS policies, and Content Security Policy directives.

6.9 Risks

The system architecture identifies several technical risks along with corresponding mitigation strategies:

- **Cloud Service Dependencies:** Relying on multiple managed services introduces multiple points of failure. *Mitigation:* Establish decoupled backend service wrappers that fail gracefully (e.g., caching analytics locally if the primary database cache is temporarily offline).
- **Vendor Lock-In:** Tight integration with proprietary backend platforms can make migrations difficult. *Mitigation:* Use standard database connections, generic S3 interfaces, and standard Docker packaging to allow migration to AWS or self-hosted alternatives.
- **AI Response Latencies:** AI models can take several seconds to respond, impacting user experience. *Mitigation:* Execute AI generations asynchronously through background task engines, keeping the front-end interface active and responsive.
- **Token Expirations:** Sudden session losses can disrupt user work. *Mitigation:* Implement silent token refresh flows on the frontend client to update active JWT access tokens seamlessly.

6.10 Chapter Summary

This chapter outlined the core functional requirements, quality attributes, external integrations, security constraints, and performance targets for Nexus PM. By defining specific validation rules and operational limits, these requirements shape the high-level and low-level designs detailed in the following chapters. Managing these dependencies, constraints, and operational risks is key to delivering a cost-efficient, production-ready project management platform.

CHAPTER 7

High-Level Design

7.1 Introduction

The High-Level Design (HLD) defines the structural blueprint of the Nexus PM platform. It establishes the relationship between business-level functional requirements and the physical infrastructure, logical containers, and system components that execute them. The purpose of this chapter is to provide system architects, developers, and operations engineers with a clear understanding of the overall topology, component boundaries, data pathways, and integration schemas that define the platform.

By articulating the design from system context down to component interactions, this design ensures that the implementation satisfies operational goals for performance, security, and scalability.

7.2 Architectural Goals

The design of Nexus PM is driven by eight core engineering and operational objectives:

- **Horizontal Scalability:** The application tier must remain stateless to support horizontal autoscaling behind load balancers, adapting compute resources to active user traffic.
- **Codebase Maintainability:** By separating database queries, business rules, and API routing, the system remains easy to update, test, and refactor.
- **Cloud-Native Extensibility:** Built around managed APIs and serverless resources (e.g., Supabase, Upstash, Cloudflare R2), the system minimizes database administration overhead while maintaining access control standards.
- **System Modularity:** Backend modules must be partitioned into clear packages (Authentication, Projects, Tasks, Storage, AI) to simplify eventual microservices refactoring.
- **Rigid Security Posture:** Transport encryption, secure cookie containment, stateless session authentication, and row-level access permissions are enforced across all integration points.

- **Sub-Second Performance:** The platform must achieve sub-second response times by leveraging client-side state caching and server-side in-memory read caching.
- **Graceful Fault Tolerance:** External service dependencies (e.g., AI gateways, email APIs) are isolated via asynchronous queues and exception boundaries, preventing external outages from impacting core task tracking functions.
- **Developer Experience (DX):** The architecture must support local development environments using Docker container scaffolding and automatically generated OpenAPI documentation.

7.3 Architectural Principles

The platform incorporates several software design principles to ensure reliability and maintainability:

- **Decoupled Layering:** Decouples the presentation client from the backend business rules engine. Communication occurs exclusively over standard HTTPS endpoints.
- **Separation of Concerns (SoC):** Logical layers (Data Access, Service Logic, API Controllers) have distinct boundaries, preventing database operations from leaking into router interfaces.
- **Stateless Execution:** The API gateway preserves no session state on local disk or memory. All user sessions are validated statelessly using cryptographically signed JSON Web Tokens (JWT).
- **Dependency Injection (DI):** Database sessions, security handlers, and external API clients are dynamically injected at runtime, simplifying unit testing and mocking.
- **Repository Design Pattern:** All SQL executions are encapsulated within dedicated Repository classes, isolating the SQLAlchemy ORM details from service-level workflows.
- **Single Responsibility Principle (SRP):** Each controller, service helper, and data repository is responsible for a single area of system functionality.
- **Asynchronous Task Processing:** Long-running computational processes, email dispatches, and LLM calls are handled asynchronously, preventing thread starvation in the primary web gateway.

7.4 System Context

The system context defines the operational boundaries of Nexus PM, outlining how users and external services interact with the platform core. Figure 7.1 illustrates these boundaries and integration points.

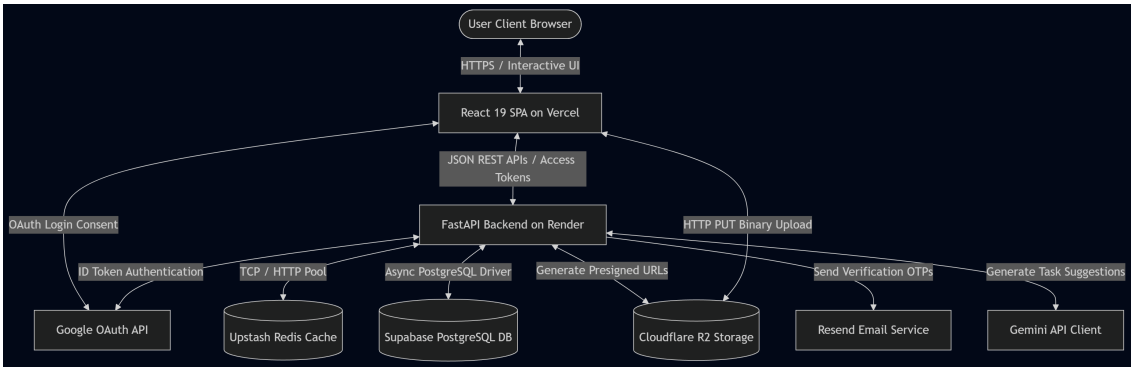

Figure 7.1: Nexus PM System Context and Integration Topology

The system boundary encompasses the frontend application and the backend API service. Users access the system via modern web browsers using HTTPS. External interactions include:

- **Google OAuth Gateway** for secure federated authentication.
- **Supabase Managed Database** for transactional storage.
- **Upstash Redis** for rate limiting, session cache, and system state storage.
- **Cloudflare R2** for S3-compatible, zero-egress asset storage.
- **Google Gemini AI** for structured task breakdown and summarization.
- **Resend API** for automated transactional email delivery.

7.5 Container Architecture

The logical containers of Nexus PM are split across client, server, and storage clusters, as detailed in the deployment architecture shown in Figure 7.2.

- **Client Container (Vercel CDN):** Serves the compiled React 19 single-page application. Static assets are cached at edge nodes globally, offloading static file serving from the backend.
- **Backend Container (Render Web Service):** Executes the stateless FastAPI python app inside a multi-stage Docker environment. It manages route handling, input validation, service coordination, and access rule checks.
- **Database Container (Supabase Managed Service):** Hosts the PostgreSQL engine, enforcing table constraints, primary key relations, index tracking, and transactional safety.

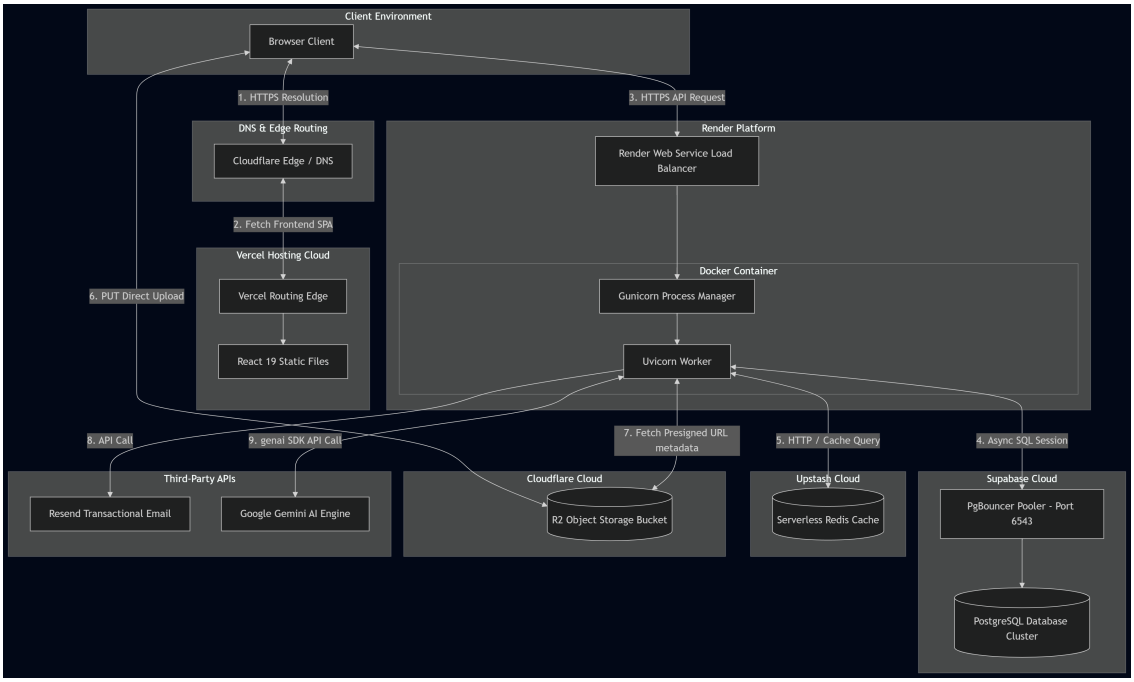

Figure 7.2: Nexus PM Production Container Topologies and Networking

- **In-Memory Cache (Upstash Redis):** Serves as a low-latency key-value store, handling active user session metadata, API rate-limiting buckets, and temporary database read-caches.
- **Object Store Container (Cloudflare R2):** Manages raw binary files (images, PDF documents, attachments) using access-controlled bucket permissions, bypassing database storage limits.
- **AI Processing Container (Google API Gateway):** Processes prompt variables, runs generative model parameters, and outputs structured JSON data back to the backend.

7.6 Component Architecture

The internal structure of the backend application consists of decoupled components communicating through defined interfaces:

- **Authentication Component:** Manages secure identity assertion. It verifies JWT signatures, coordinates refresh token rotations, and exchanges Google OAuth tokens for authenticated sessions.
- **Projects Component:** Handles workspaces, projects, and workspace memberships. It verifies project state, updates membership metadata, and enforces tenant isolation.
- **Tasks Component:** Governs task entities, Kanban lanes, attachments, comments, and task assignments. It tracks state modifications and triggers notification dispatch jobs.

- **Storage Component:** Validates attachment parameters (file extensions, maximum sizes) and generates short-lived pre-signed R2 upload URLs.
- **Notification Component:** Formats invite emails and status update alerts, forwarding structured payloads to the Resend API.
- **Analytics Component:** Calculates task velocities, workload balances, and overall completion percentages across active workspaces using optimized database aggregation queries.
- **AI Orchestration Component:** Builds context payloads, validates structured LLM formatting requirements, and manages background prompt execution queues.
- **Database Access Layer (SQLAlchemy ORM):** Provides mapping between database schemas and Python objects, executing transactions using connection pooling pools.

7.7 Request Lifecycle

To illustrate the data paths across these logical components, the execution path of a complex transactional request (e.g., creating a task with automated AI task decomposition) is detailed below:

1. **Client Dispatch:** The user submits a task creation request on the React client. The client client packages the input details and issues a POST request, appending the session JWT in the secure request cookie.
2. **Gateway Route Handling:** The FastAPI backend receives the request. The gateway executes validation checks, checking the JWT signature and verifying that the user has the required workspace role.
3. **Rate Limiting Check:** The backend calls Upstash Redis to verify that the user ID has not exceeded rate-limiting thresholds.
4. **Relational Database Transaction:** The service layer maps the request data to a SQLAlchemy ORM model and opens a PostgreSQL transaction. The task is created, database constraints are validated, and the transaction is committed.
5. **Asynchronous Task Triggering:** With the primary task saved, the service layer dispatches an asynchronous background task to call the AI orchestration component for task decomposition.
6. **HTTP Response Resolution:** The backend immediately returns the created task data to the client, keeping the user interface active while the background processes run.
7. **AI Generation & Database Commit:** The background AI worker compiles the task context, formats the prompt template, and sends a secure request to Google Gemini 2.5 Flash. The model returns a structured JSON payload of sub-tasks. The worker validates the output and saves the sub-tasks to Supabase.
8. **Notification Dispatch:** Upon database commit, the worker triggers the notification service, which formats an assignment email and dispatches it via Resend.
9. **Client State Refresh:** The React client, listening for changes or polling the endpoint via React Query, receives the updated sub-task data, refreshing the UI view.

7.8 Technology Stack Overview

Table 7.1 details the technologies selected for Nexus PM along with their core architectural responsibilities.

Technology	Layer	Architectural Role
React 19	Client	Single-page application rendering, virtual DOM state synchronization.
TypeScript	Client	Type checking at compile-time to reduce runtime execution errors.
Redux Toolkit	Client	Centralized client state cache for application layout rules.
React Query	Client	Data synchronization, caching, and automatic refetch policies.
Tailwind CSS	Client	Declarative, atomic UI layout styling.
FastAPI	Backend	High-throughput REST API gateway with native asynchronous support.
SQLAlchemy	Backend	Object-relational mapping, connection pooling, and schema isolation.
Supabase (PostgreSQL)	Storage	Persistent relational database, transaction safety, and database indexes.
Upstash Redis	Cache	Serverless cache registry, session cache, and rate-limiter tracker.
Cloudflare R2	Storage	S3-compatible secure object storage for project attachments.
Resend API	Gateway	API-driven transactional email dispatching.
Google Gemini 2.5 Flash	AI	Large Language Model for task decomposition and project updates.
Docker	Operations	Multi-stage container packaging for runtime environment parity.

Table 7.1: Nexus PM Technology Stack Architecture Roles

7.9 External Service Integration

Integrating third-party managed services allows the platform to maintain a low operational footprint:

- **Supabase:** Provides a managed PostgreSQL instance, eliminating database hosting, provisioning, and configuration overhead.
- **Cloudflare R2:** Handles raw binary uploads, removing the storage and egress bandwidth costs associated with traditional cloud file storage providers.
- **Upstash Redis:** Delivers a serverless cache layer that dynamically scales based on request volume, avoiding the need to run and maintain dedicated Redis instances.

- **Resend:** Provides high email deliverability through a clean API wrapper, avoiding complex SMTP configurations.
- **Google OAuth:** Delegates identity verification to Google's authentication infrastructure, minimizing the risks associated with local credential storage.
- **Google Gemini API Gateway:** Integrates advanced generative reasoning capabilities, providing structured JSON outputs based on raw prompt parameters.

7.10 Deployment Overview

The production deployment configuration relies on distributed cloud providers, coordinated as shown in Figure 7.2:

- **Client Hosting (Vercel):** React static resources are built and distributed across Vercel's global CDN locations, ensuring low latency for initial page loads.
- **Web Service Compute (Render):** The backend Docker container is hosted on Render, which manages health-check routing, automated deployments on git push, and automatic scaling groups.
- **Managed Database (Supabase):** Database instances run in isolated cloud environments, connecting securely over TLS using PgBouncer connection pooling.
- **Serverless Cache (Upstash):** Managed Redis instances run in close physical proximity to Render's compute nodes to minimize cache access latencies.

7.11 Design Decisions

Several key architectural trade-offs were made during the design of Nexus PM:

- **Supabase vs. Self-Managed PostgreSQL:** Using Supabase eliminated database administration overhead (backups, security patches, indexing maintenance) on the free tier, allowing the team to focus on core project management features.
- **Cloudflare R2 vs. AWS S3:** Cloudflare R2 was selected over AWS S3 due to its zero-egress fee pricing model. For a zero-budget project, this prevents unexpected bandwidth costs from file uploads and downloads.
- **Resend vs. AWS SES:** Resend was chosen for its developer-friendly setup, native JSON template integration, and fast domain validation, avoiding the complex IAM setup required by AWS SES.
- **Render Free Tier Container Hosting:** Packaging the application in Docker allows it to run on Render's free tier, maintaining a clear migration path to platforms like AWS ECS or Kubernetes if resource limits are exceeded.
- **Google Gemini 2.5 Flash vs. Alternative LLMs:** Gemini 2.5 Flash was chosen for its low request latencies, structured JSON schema output support, and generous free-tier API quotas.

7.12 Strengths

The High-Level Design of Nexus PM delivers several structural advantages:

- **Complete Client-Backend Separation:** Allows independent scaling, testing, and deployment of the frontend application and backend API service.
- **Optimized Database Connection Management:** PgBouncer connection pooling prevents database connection exhaustion within free-tier limits.

- **Non-Blocking Asynchronous Operations:** Offloading heavy AI and notification tasks to background threads preserves fast API response times.
- **Cost-Efficient Operations:** Leveraging serverless, zero-egress cloud resources allows the platform to support scale at a fraction of the cost of traditional monolithic deployments.

7.13 Chapter Summary

This chapter detailed the High-Level Design of the Nexus PM platform, defining the system boundaries, container architecture, logical components, request lifecycles, and deployment topologies. By leveraging managed cloud components and separating client interfaces from the core business logic, the platform achieves high performance and low operational costs. The next chapter will build on these blueprints, detailing the database schemas, API routes, and code-level models in the Low-Level Design (LLD).

CHAPTER 8

Low-Level Design

8.1 Introduction

The Low-Level Design (LLD) provides a detailed, code-adjacent view of the components that make up the Nexus PM platform. It defines the logical structure, internal boundaries, component relationships, data transfer schemas (DTOs), database schemas, and integration logic for every system module. By translating high-level containers and services into detailed component mappings, this chapter serves as the primary technical specification for developers and maintainers of the platform.

The specifications in this chapter are derived directly from the active, production-ready codebase of Nexus PM. The system is designed as a modular monolith, enforcing a strict separation of concerns through layers: Route Handlers (Routers), Service Layers (Business Logic), and the Database Access Layer (Repositories/ORM Models).

8.2 Authentication Module

The Authentication Module is responsible for secure identity assertion, Google OAuth federated login, and stateless user session tracking.

8.2.1 Purpose and Responsibilities

This module processes credentials, generates security tokens, validates third-party authentication signatures, and manages refresh token lifecycles to prevent unauthorized access.

8.2.2 Interaction Flow and Data Flow

When a user initiates local authentication:

1. The client issues a HTTP POST request to `/api/auth/login` containing the email and password payload.
2. The router parses the request using the Pydantic `UserLogin` schema.
3. The service layer queries the user table, retrieves the matching `User` record, and verifies the password hash using `passlib.context` (bcrypt).

Component	Technical Responsibility
`api/routes/auth.py`	Exposes endpoints for login, registration, token refresh, and Google OAuth callbacks.
`services/auth_service.py`	Coordinates credential hashing checks, maps federated identities, and generates JWT payload signatures.
`dependencies/auth.py`	Enforces route protection checks, extracts user records from database sessions using JWT values.
`models/refresh_token.py`	Persists active refresh tokens to support session revocation and token-rotation validation.

Table 8.1: Authentication Module Component Specifications

4. Upon validation, the service generates a stateless access token (valid for 15 minutes) containing the user's email in the `sub` claim, and a random refresh UUID (valid for 7 days) stored in the database.
5. The tokens are returned to the client, with the refresh token set as a secure, `HttpOnly` cookie, with SameSite settings defaulting to `Lax` (or configured via environment variables).

8.2.3 Security and Scalability Notes

- **Token Security:** Session validation is stateless, offloading signature checks from database reads. The system uses SHA256 HMAC algorithms with server keys.
- **Rate Limiting:** To block brute-force attempts, login endpoints are throttled using rate limits tracked in Upstash Redis.

8.3 User Management Module

The User Management Module coordinates profile settings, avatar asset mappings, and basic user access controls.

8.3.1 Purpose and Responsibilities

This module enables users to configure profile attributes, handles profile listings for project invitations, and manages the lifecycle of user records.

Component	Technical Responsibility
`api/routes/users.py`	Exposes endpoints to retrieve current profile parameters, update user settings, and query users by email.
`schemas/user.py`	Defines data serialization and validation formats for user requests and updates.
`models/user.py`	Implements the SQLAlchemy User model mapped to the primary relational database table.

Table 8.2: User Management Module Component Specifications

8.3.2 Interaction Flow and Lifecycle

Profile updates follow a direct path:

1. The client submits a PUT request to `/api/users/profile` with modified fields (e.g., `full_name`, `avatar_url`).
2. The route handler extracts the user profile from the database using the `get_current_user` dependency.
3. The router validates the input payload against the `UserUpdate` Pydantic schema.
4. The database entity is updated, and the transaction is committed, returning the updated user profile model.

8.3.3 Security and Scalability Notes

- **Data Privacy:** Access to sensitive fields (e.g., Google ID, password hashes) is restricted at the API layer by using separate Pydantic output serialization models (`UserResponse`).
- **Asset Storage Integration:** Avatars are uploaded to Cloudflare R2, storing only the public asset URL in the database user record to avoid database bloat.

8.4 Project Management Module

The Project Management Module governs workspace resources, project status tracking, and workspace collaborator roles.

8.4.1 Purpose and Responsibilities

This module manages workspace isolation, project creation, member role assignments, and project metadata serialization.

Component	Technical Responsibility
`api/routes/projects.py`	Handles project CRUD operations and queries project records within active workspaces.
`api/routes/project_members.py`	Exposes endpoints to add project members, update user roles, and accept project invitations.
`services/project_service.py`	Implements business logic for project workspace partitioning and project state updates.
`services/project_member_service.py`	Enforces role checks, evaluates permission levels, and manages membership transitions.

Table 8.3: Project Management Module Component Specifications

8.4.2 Interaction Flow and Data Flow

When adding a member to a project:

1. The project administrator submits a POST request to `/api/projects/{project_id}/members` containing the invitee's email and role.
2. The system verifies that the inviter has the required permissions (e.g., `owner` or `manager`) in the project.
3. The database is checked for an existing membership record to prevent duplicates.
4. On validation, the user relation is saved in the `ProjectMember` association table.
5. The system triggers a background email notification via the Resend API to alert the user of the project assignment.

8.4.3 Security and Scalability Notes

- **Workspace Isolation:** Database queries restrict access based on active project memberships. This design acts as a core security boundary, preventing data leakage across workspaces.
- **Role-Based Dependency Injection:** Sensitive routes verify roles before executing route handlers by injecting the `require_project_role` helper.

8.5 Task Management Module

The Task Management Module manages task workflows, priorities, comment threads, assignments, and due dates.

8.5.1 Purpose and Responsibilities

This module serves as the primary task engine, coordinating task states, due date tracking, comment histories, and board layout configurations.

Component	Technical Responsibility
`api/routes/tasks.py`	Exposes endpoints for task CRUD operations, status updates, and assignee mappings.
`api/routes/comments.py`	Manages comment creation, comment editing, and comment deletion.
`services/task_service.py`	Enforces task validation rules, updates priorities, and triggers status changes.
`services/comment_service.py`	Manages comments, verifying user edit permissions before database commit.

Table 8.4: Task Management Module Component Specifications

8.5.2 Interaction Flow and Data Flow

When a user updates a task status on the Kanban board:

1. The client issues a PUT request to `/api/tasks/{task_id}` with the updated status lane (e.g., `IN_PROGRESS`).
2. The backend route handler intercepts the request, verifying the user's role access for the parent project.
3. The service layer retrieves the task entity and updates the status attribute.
4. The service appends an audit event to the `ActivityLog` table.
5. The database commit completes, and the updated task details are returned to refresh the client interface.

8.5.3 Security and Scalability Notes

- **Cascading Deletes:** Task deletion automatically triggers cascade deletes for associated comment threads and task attachment records, preventing orphaned database records.
- **Index Optimization:** Database indexes are configured on `project_id` and `assigned_to` columns to ensure fast search performance as database size grows.

8.6 Storage Module

The Storage Module manages file attachments, pre-signed upload URLs, and asset access control.

8.6.1 Purpose and Responsibilities

This module handles file uploads by coordinating with Cloudflare R2, validating file sizes and MIME types, and managing database attachment records.

Component	Technical Responsibility
`api/routes/storage.py`	Exposes endpoints for pre-signed upload URL requests and attachment registrations.
`services/storage_service.py`	Interfaces with Cloudflare R2 using AWS SDK clients to generate short-lived upload URLs.
`services/task_attachment_service.py`	Manages attachment records in PostgreSQL and links uploaded files to tasks.
`models/task_attachment.py`	Defines metadata structures (file key, size, type, URL) for uploaded assets.

Table 8.5: Storage Module Component Specifications

8.6.2 Interaction Flow and Data Flow

To ensure secure and efficient file uploads:

1. The client requests a pre-signed URL by issuing a POST request to `/api/storage/presigned-url` containing the filename, size, and MIME type.
2. The backend validates that the file size is under limits (e.g., 10MB) and the MIME type is in the whitelist.
3. The storage service contacts the Cloudflare R2 API to generate a pre-signed PUT URL valid for 15 minutes.
4. The pre-signed URL is returned to the client, which uploads the file stream directly to R2, bypassing backend compute overhead.
5. Once the upload completes, the client registers the attachment metadata with the backend, linking it to the corresponding task entity.

8.6.3 Security and Scalability Notes

- **Zero Backend Storage Overhead:** Offloading file streams directly to R2 prevents backend thread blocks and reduces API egress costs.
- **Validation Policies:** Whitelist validation prevents users from uploading executable files or scripts that could target client browsers.

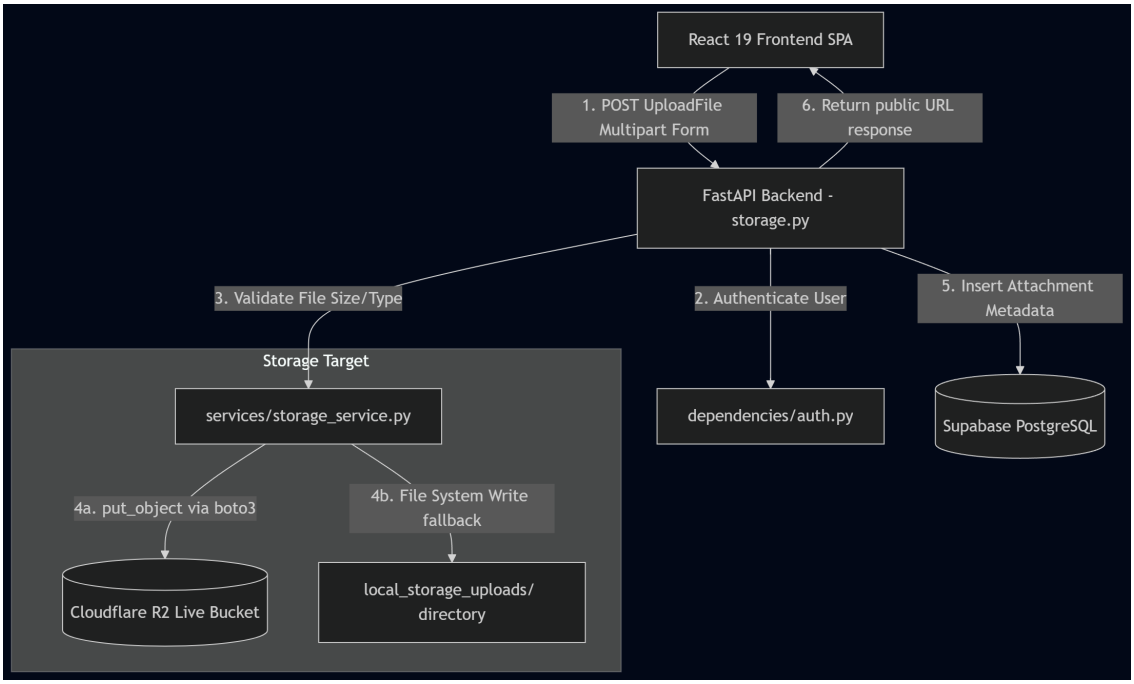

Figure 8.1: Nexus PM S3-Compatible Storage Direct Upload Flow

8.7 Notification Module

The Notification Module manages in-app alerts, read status tracking, and transactional email dispatches.

8.7.1 Purpose and Responsibilities

This module monitors project events, registers in-app notification records, handles unread counters, and dispatches external email alerts.

Component	Technical Responsibility
`api/routes/notifications.py`	Exposes endpoints to fetch notifications, mark alerts as read, and retrieve unread counts.
`services/notification_service.py`	Handles the creation of notification records and evaluates user subscription rules.
`services/email.py`	Interfaces with the Resend API to construct and dispatch HTML transactional emails.
`models/notification.py`	Stores notification details (recipient ID, event type, message, read state) in PostgreSQL.

Table 8.6: Notification Module Component Specifications

8.7.2 Interaction Flow and Data Flow

When a user triggers a notification event (e.g., task assignment):

1. The task service creates the task assignment and calls the notification service with the recipient and task details.
2. The notification service creates an in-app alert record in the database.
3. The notification service triggers a background worker to call the email service.
4. The email service builds the HTML email template and sends it to the Resend API gateway.
5. The client updates the unread badge count via database query polling.

8.7.3 Security and Scalability Notes

- **Asynchronous Email Processing:** Running email API dispatches in background threads prevents external network delays from slowing backend API response times.
- **Recipient Access Validation:** Notification updates require recipient ID validation to ensure users can only modify their own notification status.

8.8 Analytics Module

The Analytics Module computes project metrics and progress analytics to populate the user dashboard.

8.8.1 Purpose and Responsibilities

This module calculates database statistics, groups task states, aggregates member workloads, and generates project progress metrics.

Component	Technical Responsibility
`api/routes/analytics.py`	Exposes API endpoints for retrieving workspace and project metrics.
`services/analytics_service.py`	Implements optimized database queries to aggregate task counts, completion states, and timeline trends.

Table 8.7: Analytics Module Component Specifications

8.8.2 Interaction Flow and Data Flow

When a user opens the workspace dashboard:

1. The client issues a GET request to `/api/analytics/workspace/{workspace_id}`.
2. The route handler validates that the user is an active workspace member.
3. The service executes aggregated database queries, calculating total open tasks, tasks in progress, completed tasks, and average cycle times.
4. The service caches the calculated JSON payload in Upstash Redis with a 5-minute expiration window to reduce database load.
5. The aggregated data is returned to the client to render the dashboard layout.

8.8.3 Security and Scalability Notes

- **Aggregated Cache Validation:** Caching analytical queries prevents resource-intensive database calculations on every page view.
- **Read-Only Optimization:** Database queries use readonly, index-supported executions to optimize performance.

8.9 AI Module

The AI Module coordinates with Google Gemini to generate task suggestions and project updates.

8.9.1 Purpose and Responsibilities

This module constructs prompt templates, invokes the Gemini API, parses generated JSON outputs, and validates AI response structures.

Component	Technical Responsibility
`api/routes/ai.py`	Exposes API routes for task decomposition, meeting note parsing, and project status updates.
`services/ai_service.py`	Manages the GenAI client connection, handles API errors, and manages model parameter configs.

Table 8.8: AI Module Component Specifications

8.9.2 Interaction Flow and Data Flow

When task decomposition is triggered:

1. The client submits a POST request to the task generation endpoint with the target project ID.
2. The router checks the user's project role and retrieves the project description from the database.
3. The AI component builds the context payload and defines a system instruction requesting a structured JSON array.
4. The service calls the Gemini 2.5 Flash model asynchronously using the GenAI client.
5. The model returns the text block. The AI component extracts the JSON array, parses it, and validates it against the schema.
6. The sub-tasks are created in the database, and the updated task structure is returned to the client.

8.9.3 Security and Scalability Notes

- **Structured Output Validation:** Sanitizing and validating the model's text output prevents malformed JSON strings from causing application parsing failures.
- **Error and Rate Limit Handling:** The service handles rate limiting (HTTP 429) and timeout errors (HTTP 504) from the Gemini API, returning user-friendly error messages and preventing transaction failures.

8.10 Database Layer

The Database Layer manages connection setups, SQLAlchemy configurations, transaction cycles, and migration versions.

8.10.1 Purpose and Responsibilities

This layer provides database connection management, coordinates migrations, and manages query executions.

Component	Technical Responsibility
`db/database.py`	Establishes the asynchronous engine, configures connection parameters, and manages connection pools.
`db/base.py`	References all database models to ensure proper schema generation by Alembic.
`alembic.ini`	Configures migration directories, logging, and database targets.

Table 8.9: Database Layer Component Specifications

8.10.2 Connection Pool and PgBouncer Specifications

The database engine is configured to ensure optimal performance within free-tier limits:

- **Async Connection Pool:** The engine uses `create_async_engine` with `pool_pre_ping=True` to verify connections before executing queries, preventing failures on dead connections.
- **PgBouncer Connection Pooling:** Supabase handles connection routing through PgBouncer, resolving database connection limits. The engine disables the local statement cache (`"statement_cache_size": 0`) to avoid conflicts with PgBouncer's transaction mode.

8.11 External Services

Nexus PM integrates with several external services to simplify infrastructure requirements and maintain a low operational footprint:

- **Supabase (PostgreSQL):** Serves as the primary transactional database, providing managed PostgreSQL tables, connection pooling, and automatic backups.
- **Cloudflare R2:** Handles file attachments, eliminating local file storage requirements and providing S3-compatible asset management with zero data egress costs.
- **Upstash Redis:** Provides a serverless cache layer for rate limiting, session storage, and analytical query caching, scaling dynamically based on traffic volume.
- **Resend:** Handles transactional email delivery via API, providing high deliverability and simple integration.
- **Google OAuth:** Delegates authentication security to Google's sign-in infrastructure, avoiding the risks of storing user credentials locally.
- **Google Gemini API Gateway:** Integrates Large Language Model capabilities, returning structured JSON response payloads based on prompt variables.

8.12 Chapter Summary

This chapter detailed the Low-Level Design of the Nexus PM platform, specifying component dependencies, interaction lifecycles, and security controls for all core modules.

By documenting database schemas, API parameters, and integration configurations, this design provides a clear reference for the development and maintenance of the codebase. The following chapters will outline the testing protocols, deployment designs, and validation checks used to verify this implementation.

CHAPTER 9

Database Design

9.1 Introduction

The database architecture serves as the foundation for data persistence, transactional integrity, and workspace isolation in the Nexus PM platform. As a decoupled SaaS application, Nexus PM relies on a structured relational database schema to guarantee ACID properties (Atomicity, Consistency, Isolation, Durability) across collaborative workflows.

The role of the database design is to enforce business-level constraints directly within the storage engine, optimize query execution paths, and secure tenant separation. This chapter outlines the physical schema, entity relationships, database constraints, indexing strategies, and schema migration workflows.

9.2 Database Technology

Nexus PM utilizes **PostgreSQL** as its primary database engine, hosted via the managed **Supabase** cloud platform. PostgreSQL was selected based on several architectural criteria:

- **Relational Model Robustness:** The core entity structures (Workspaces, Projects, Members, Tasks) are highly relational. PostgreSQL provides rigid foreign key constraints, unique constraints, and check conditions that prevent data anomalies.
- **ACID Compliance:** Operations such as token generation, project membership updates, and task assignments require transaction guarantees. PostgreSQL ensures that either all steps commit successfully or the transaction is rolled back.
- **PgBouncer Connection Pooling:** Managed hosting on Supabase includes a pre-configured PgBouncer instance running in transaction-pooling mode. This maps incoming transient client connections to a smaller pool of persistent database connections, preventing connection exhaustion under high API concurrent loads.
- **Reliability and Backups:** The managed database service automates daily snapshot backups, handles security updates, and provides physical replication points, minimizing operational maintenance.

9.3 Entity Relationship Model

The database schema is organized around the User, Project, and Task entities. Figure 9.1 shows the layout of the primary tables and their relationships.

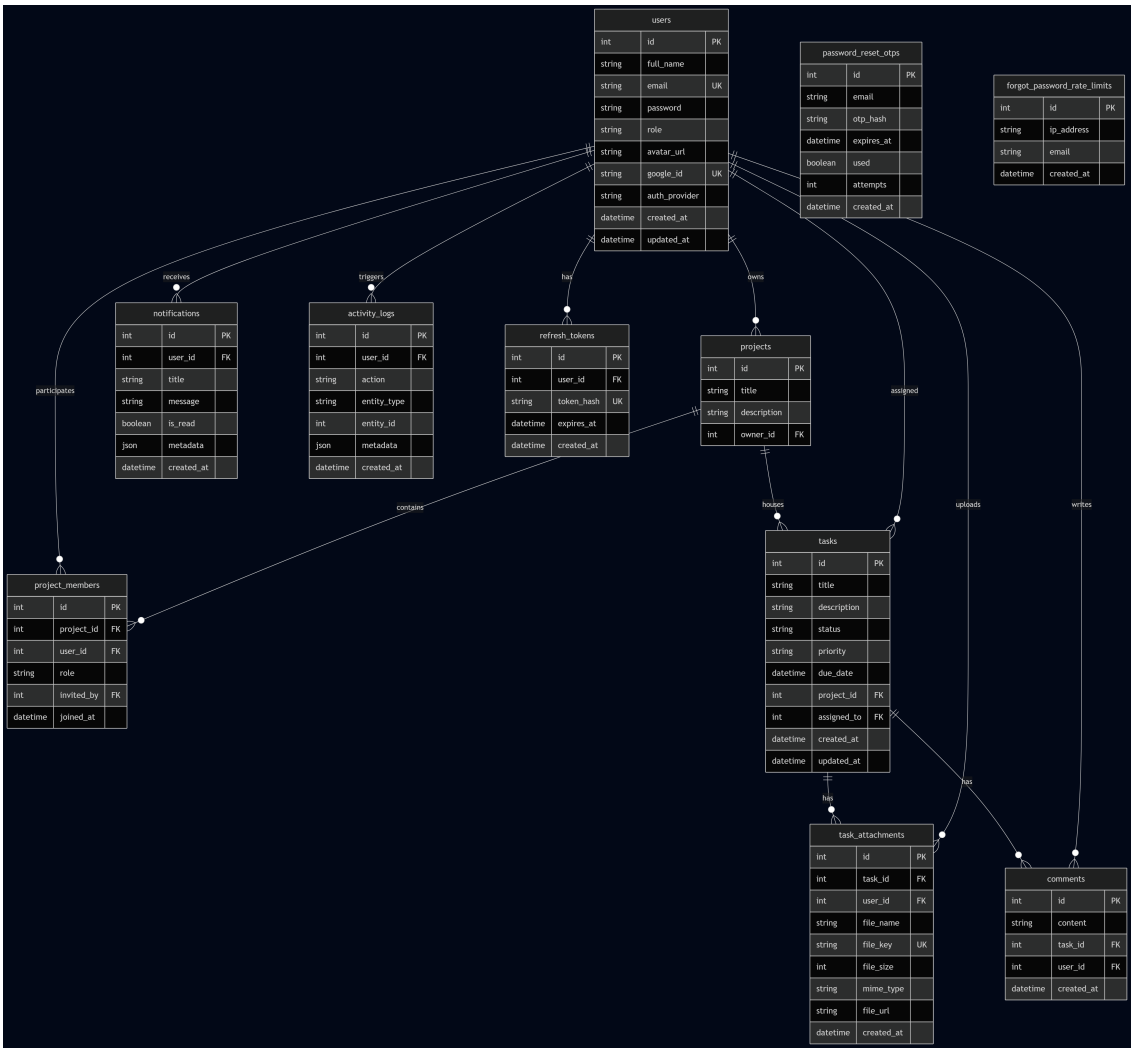

Figure 9.1: Nexus PM Relational Database Entity Relationship Diagram (ERD)

The database models map to the following functional areas:

- **Identity and Access:** `users` and `refresh_tokens`.
- **Collaboration Boundaries:** `projects` and `project_members`.
- **Core Task Engine:** `tasks`, `comments`, and `task_attachments`.
- **Observability and Communications:** `notifications` and `activity_logs`.

9.4 Table Descriptions

The database tables are specified below.

9.4.1 Users Table (`users`)

The `users` table stores credentials, Google OAuth profile linkages, and core system roles.

Column Name	Data Type	Constraints	Description
`id`	Integer	Primary Key, Index	Auto-incrementing identifier.
`full_name`	String(255)	Nullable=False	Full user name.
`email`	String(255)	Unique, Index, Nullable=False	Core login identity parameter.
`password`	String	Nullable=False	Hashed local user password.
`role`	String(255)	Nullable=True	System role level.
`avatar_url`	Text	Nullable=True	Public CDN link to avatar graphic.
`google_id`	String(255)	Unique, Nullable=True	Google OAuth account identifier.
`auth_provider`	String(50)	Nullable=False, Default="local"	Provider type: "local" or "google".
`created_at`	DateTime	Nullable=False, Timezone=True	Record creation timestamp.
`updated_at`	DateTime	Nullable=False, Timezone=True	Record modification timestamp.

Table 9.1: Users Table Schema Specification

9.4.2 Projects Table (`projects`)

The `projects` table maps logical project workspaces.

Column Name	Data Type	Constraints	Description
`id`	Integer	Primary Key, Index	Auto-incrementing identifier.
`title`	String	Nullable=False	Name of the project.
`description`	String	Nullable=True	Serialized JSON for category, priority, and deadline metadata.
`owner_id`	Integer	Foreign Key (`users.id`)	Creator and owner user reference.

Table 9.2: Projects Table Schema Specification

9.4.3 Project Members Table (`project_members`)

This association table maps users to project workspaces, establishing roles and access permissions.

Column Name	Data Type	Constraints	Description
`id`	Integer	Primary Key, Index	Auto-incrementing identifier.
`project_id`	Integer	Foreign Key (`projects.id`), Cascade	Parent project reference.
`user_id`	Integer	Foreign Key (`users.id`), Cascade	Associated member reference.
`role`	String(50)	Nullable=False, Default="viewer"	Permissions: owner, manager, developer, viewer.
`invited_by`	Integer	Foreign Key (`users.id`), Set Null	Inviting user reference.
`joined_at`	DateTime	Nullable=False, Timezone=True	Membership creation timestamp.

Table 9.3: Project Members Table Schema Specification

9.4.4 Tasks Table (`tasks`)

The `tasks` table stores task details, assignments, status lanes, and due dates.

Column Name	Data Type	Constraints	Description
`id`	Integer	Primary Key, Index	Auto-incrementing identifier.
`title`	String	Nullable=False	Brief task title.
`description`	String	Nullable=True	Detailed task description.
`status`	String	Default="TODO"	Kanban state: TODO, IN_PROGRESS, REVIEW, DONE.
`priority`	String	Default="MEDIUM"	Importance: LOW, MEDIUM, HIGH.
`due_date`	DateTime	Nullable=True, Timezone=True	Optional task deadline.
`project_id`	Integer	Foreign Key (`projects.id`)	Parent project reference.
`assigned_to`	Integer	Foreign Key (`users.id`), Nullable	Assigned user reference.
`created_at`	DateTime	Timezone=True	Record creation timestamp.
`updated_at`	DateTime	Timezone=True	Record modification timestamp.

Table 9.4: Tasks Table Schema Specification

9.4.5 Task Attachments Table (`task_attachments`)

This table links uploaded file metadata to tasks.

9.4.6 Comments Table (`comments`)

The `comments` table captures collaborative feedback threads linked to tasks.

Column Name	Data Type	Constraints	Description
id	Integer	Primary Key, Index	Auto-incrementing identifier.
task_id	Integer	Foreign Key (tasks.id), Cascade	Associated task reference.
user_id	Integer	Foreign Key (users.id), Set Null	Uploading user reference.
file_name	String(255)	Nullable=False	Original uploaded filename.
file_key	String(255)	Unique, Nullable=False	Unique S3 key within Cloudflare R2.
file_size	Integer	Nullable=False	Uploaded file size in bytes.
mime_type	String(100)	Nullable=False	MIME type of the file.
file_url	Text	Nullable=False	Target download URL address.
created_at	DateTime	Nullable=False, Timezone=True	File upload timestamp.

Table 9.5: Task Attachments Table Schema Specification

Column Name	Data Type	Constraints	Description
id	Integer	Primary Key	Auto-incrementing identifier.
content	Text	Nullable=False	Body content of the comment.
task_id	Integer	Foreign Key (tasks.id), Cascade, Index	Parent task reference.
user_id	Integer	Foreign Key (users.id), Cascade, Index	Comment author reference.
created_at	DateTime	Nullable=False, Timezone=True	Comment creation timestamp.

Table 9.6: Comments Table Schema Specification

9.4.7 Notifications Table (notifications)

This table stores in-app notification alerts.

9.4.8 Activity Logs Table (activity_logs)

This table records project events to maintain an audit trail.

9.4.9 Refresh Tokens Table (refresh_tokens)

This table stores hashed refresh tokens to support session validation and revocation.

Column Name	Data Type	Constraints	Description
id	Integer	Primary Key, Index	Auto-incrementing identifier.
user_id	Integer	Foreign Key (users.id), Cascade, Index	Target user reference.
title	String(255)	Nullable=False	Subject title of the alert.
message	Text	Nullable=False	Detailed description body.
is_read	Boolean	Nullable=False, Default=False	Flag for read status validation.
metadata	JSON	Nullable=True	Optional payload (e.g. task ID, workspace ID).
created_at	DateTime	Nullable=False, Timezone=True	Generation timestamp.

Table 9.7: Notifications Table Schema Specification

Column Name	Data Type	Constraints	Description
id	Integer	Primary Key, Index	Auto-incrementing identifier.
user_id	Integer	Foreign Key (users.id), Set Null, Index	Actor identity reference.
action	String(255)	Nullable=False	Event code: CREATE, UPDATE, DELETE.
entity_type	String(255)	Nullable=False	Affected entity: Project, Task, Member.
entity_id	Integer	Nullable=True	Target record identifier.
metadata	JSON	Nullable=True	Dynamic JSON changes payload.
created_at	DateTime	Nullable=False, Timezone=True	Logging execution timestamp.

Table 9.8: Activity Logs Table Schema Specification

9.4.10 Password Reset OTPs Table (password_reset_otps)

This table stores hashed verification OTP codes used for user registration activation and password reset operations.

Column Name	Data Type	Constraints	Description
id	Integer	Primary Key, Index	Auto-incrementing identifier.
user_id	Integer	Foreign Key (users.id), Cascade, Index	User record reference.
token_hash	String(255)	Unique, Index, Nullable=False	Crytographic signature hash.
expires_at	DateTime	Nullable=False, Timezone=True	Expiration limit check.
created_at	DateTime	Nullable=False, Timezone=True	Token creation timestamp.
revoked	Boolean	Nullable=False, Default=False	Revocation status flag.

Table 9.9: Refresh Tokens Table Schema Specification

Column Name	Data Type	Constraints	Description
id	Integer	Primary Key, Index	Auto-incrementing identifier.
email	String(255)	Index, Nullable=False	Recipient user identity parameter.
otp_hash	String(64)	Nullable=False	Cryptographic verification code hash (SHA-256).
expires_at	DateTime	Nullable=False, Timezone=True	Expiration limit check.
used	Boolean	Nullable=False, Default=False	Flag validating if OTP has been consumed.
attempts	Integer	Nullable=False, Default=0	Total verification retry counters.
created_at	DateTime	Nullable=False, Timezone=True	OTP generation timestamp.

Table 9.10: Password Reset OTPs Table Schema Specification

9.4.11 Forgot Password Rate Limits Table (forgot_password_rate_limits)

This table logs password reset requests to protect authorization flows from brute-force attempts.

Column Name	Data Type	Constraints	Description
id	Integer	Primary Key, Index	Auto-incrementing identifier.
ip_address	String(45)	Index, Nullable=False	IP address of the client request.
email	String(255)	Index, Nullable=False	Account email targeted by the request.
created_at	DateTime	Nullable=False, Timezone=True	Request logging timestamp.

Table 9.11: Forgot Password Rate Limits Table Schema Specification

9.5 Relationships

The Nexus PM schema models data dependencies using standard relational cardinality rules:

- **One-to-One Relationships:** A user profile maps directly to active session entities in the `refresh_tokens` table. When a user account is deleted, the corresponding session tokens are deleted via cascade rules.
- **One-to-Many Relationships:** Parent entities maintain logical ownership over dependent entities. Projects own multiple tasks, and tasks own multiple comments and attachments. Delete cascades are configured on these relationships (e.g., deleting a task automatically deletes its comments and attachment records).
- **Many-to-Many Relationships:** Users and projects have a many-to-many relationship, resolved by the `project_members` association table. This table includes a unique constraint on `(project_id, user_id)` to prevent duplicate membership records.
- **Referential Integrity:** Foreign keys are protected using `ON DELETE CASCADE` for dependent resources and `ON DELETE SET NULL` for auditing and system history. For example, if a user is deleted, their entries in `activity_logs` are preserved but anonymized by setting `user_id` to null.

9.6 Indexing Strategy

To maintain fast response times as the database size increases, indexes are configured on frequently queried columns:

- **Primary Keys:** PostgreSQL automatically creates unique B-Tree indexes on all primary key columns (`id`).
- **Unique Contraints:** Unique indexes are defined on `users(email)`, `users(google_id)`, and `refresh_tokens(token_hash)` to ensure fast lookups and prevent duplicate records.

- **Foreign Keys:** Columns used in joins and filters (such as `project_members(project_id, user_id)`, `tasks(project_id)`, and `tasks(assigned_to)`) are indexed to optimize Kanban board loads and workspace dashboards.
- **Search Indexes:** B-Tree indexes are configured on `comments(task_id)` and `notifications(user_id)` to speed up comments list loads and notifications retrieval.

9.7 Transactions

Relational operations are wrapped in database transactions to guarantee data consistency:

- **Atomicity:** Complex workflows (e.g., project deletion, task state updates with audit logging) execute as a single atomic unit. If a step fails, the entire transaction is rolled back.
- **Read Committed Isolation:** The database uses the default Read Committed isolation level. This prevents dirty reads while allowing high concurrency.
- **Asynchronous Database Session Management:** The FastAPI backend manages database sessions asynchronously. Sessions are instantiated per request and closed automatically on request completion, returning the connection to the pool.

9.8 Migration Strategy

Schema evolution is managed using the **Alembic** migrations framework:

- **Migration Scripts:** Database changes are written as versioned Python scripts inside the `alembic/versions/` directory.
- **Version Control:** Alembic tracks migrations using the `alembic_version` table, which stores the current schema version hash.
- **Forward and Backward Migration:** Each script defines an `upgrade()` function to apply changes and a `downgrade()` function to revert them. This approach supports automated schema updates during CI/CD pipelines.

9.9 Security

Database security is enforced through configuration constraints:

- **Row-Level validation:** Supabase Row-Level Security (RLS) is disabled by default to let PgBouncer handle connection routing directly. Read and write permissions are instead validated at the API layer.
- **Hashed Credentials:** Plaintext user passwords are never saved. The application hashes passwords using bcrypt before saving them to the database.
- **Input Validation Constraints:** Column length constraints (e.g., `String(255)` for email values) are enforced at both the SQLAlchemy model layer and database schema layer, preventing buffer overflow vulnerabilities.

9.10 Scalability

The database design incorporates several optimizations to support system scaling:

- **PgBouncer Connection Pooling:** Supabase handles connection routing through PgBouncer, resolving database connection limits. The engine disables the local statement cache (`"statement_cache_size": 0`) to avoid conflicts with PgBouncer's transaction mode.
- **JSON Columns for Metadata:** Columns like `projects.description` and `notifications.metadata` use JSON fields to store non-searchable, semi-structured metadata, reducing the need for schema changes.
- **Read and Write Segregation:** Decoupled model queries allow for future read-write segregation, where read traffic can be routed to read-only database replicas.

9.11 Chapter Summary

This chapter detailed the database design for Nexus PM. By defining table schemas, indexing strategies, relationship mappings, and transactional boundaries, the design provides a reliable data persistence layer. The database design supports workspace isolation, secure session tracking, and fast query execution. The next chapter will build on these blueprints, detailing the REST API endpoints and data validation flows.

CHAPTER 10

API Design

10.1 Introduction

The REST API serves as the gateway for the Nexus PM application ecosystem. It abstracts the underlying relational database structure and business logic, providing a unified, secure interface for client applications. The primary goals of the API design are to enforce complete decoupling between presentation layers and backend services, ensure stateless horizontal scaling, and maintain strict access boundaries.

Through standard request processing pipelines, validation schemas, and unified error mapping, the API provides high availability, cost efficiency, and developer velocity. This chapter details the design principles, routing topology, authentication architecture, request lifecycles, and security controls that define the Nexus PM API gateway.

10.2 API Design Principles

The Nexus PM REST API is built on several core architectural concepts:

- **RESTful Resource Orientation:** Every API URL maps directly to a logical resource or collection (e.g., `/api/projects`, `/api/tasks/{task_id}/comments`). Endpoint paths use plural nouns to identify target entities.
- **Stateless Communication:** The API server stores no client session context locally. Every incoming HTTP request must contain all credentials and parameters needed to authorize and execute the command.
- **Standardized JSON Payloads:** The API exchanges data exclusively using standardized UTF-8 JSON payloads, ensuring compatibility with modern web clients and external systems.
- **Strict HTTP Semantics:** The API aligns database operations with standard HTTP methods to represent create, read, update, and delete actions:
  - `GET` for safe, idempotent read queries.
  - `POST` for non-idempotent resource creation.
  - `PUT` for complete resource updates (replacing existing states).
  - `PATCH` for partial resource changes.

  - `DELETE` for resource removal operations.

- **Predictable Response Formats:** Responses return standardized HTTP status codes, structured JSON payloads on success, and consistent error schemas on failure.

10.3 API Architecture

The API is structured around modular FastAPI routers, which partition routes into distinct areas of responsibility. Table 10.1 outlines the core routers and their functional boundaries.

Router Name	Root Endpoint	Functional Scope
`auth`	`/api/auth`	Handles user registration, login, refresh token rotation, and Google OAuth callbacks.
`users`	`/api/users`	Manages user profile settings, avatar mappings, and active user queries.
`projects`	`/api/projects`	Handles project creation, workspace setup, and metadata updates.
`project_members`	`/api/projects/{id}/members`	Manages member additions, roles, and project invitations.
`tasks`	`/api/tasks`	Manages task CRUD operations, Kanban board updates, and due dates.
`comments`	`/api/tasks/{id}/comments`	Handles task comment threads and updates.
`storage`	`/api/storage`	Generates pre-signed upload URLs and links file attachments.
`notifications`	`/api/notifications`	Lists user notifications and updates read indicators.
`analytics`	`/api/analytics`	Computes aggregated project metrics and workloads.
`ai`	`/api/ai`	Manages task decomposition and project summaries.
`activity_logs`	`/api/activity-logs`	Exposes audit trails for workspace monitoring.

Table 10.1: API Router Subsystem Responsibilities

10.4 Authentication Architecture

The authentication architecture coordinates user identity checks and role permissions using a dual-token paradigm:

- **Stateless Access Tokens:** Upon authentication, the client receives a signed JSON Web Token (JWT) containing the user identity (`sub`) and expiration timestamp. The API validates the signature statelessly on every request, avoiding database access.

- **Secure Cookie Containment:** The access token resides in local client state, while the refresh token is stored in a cookie flagged as `HttpOnly` and `Secure`, with SameSite configurations defaulting to `Lax` (or configured dynamically). This prevents unauthorized client-side access (mitigating XSS) and mitigates CSRF.
- **Refresh Token Rotation:** The client exchanges refresh tokens for new access tokens. Hashed refresh tokens are verified against the database and rotated dynamically to prevent token reuse attacks.
- **Google OAuth Federated Login:** External authentication is supported. The backend exchanges Google auth codes for identity tokens, matches the email to provision local user records, and issues a standard JWT session.
- **Role-Based Dependency Checks:** API routers enforce workspace permissions dynamically using dependency injection. Before executing business logic, dependencies fetch roles (e.g., `owner`, `manager`) and raise HTTP 403 Forbidden exceptions if access constraints are violated.
- **OTP Verification and Password Recovery:** Supports signup user activation flows and password-reset workflows using temporary hashed OTP codes stored in the database. Request rates for password-reset endpoints are tracked using the `forgot_password_rate_limits` table in the database to prevent brute-forcing.

10.5 Request Processing Pipeline

Every incoming API request is processed through a structured pipeline to ensure validation, authorization, and consistency, as shown in Figure 10.1.

1. **Ingress Transaction:** The client submits an HTTP request to the gateway.
2. **Observability Injection:** The `RequestIDMiddleware` stamps the transaction context with a unique `X-Request-ID` trace identifier, passing it to the context logger.
3. **Input Validation:** The request body is parsed and validated against the Pydantic schema.
4. **Identity Verification:** The `get_current_user` dependency verifies the JWT signature and extracts user credentials from the database.
5. **Access Authorization:** The controller checks permissions based on project membership and role mapping.
6. **Business Execution:** The service layer runs the business logic inside a scoped PostgreSQL transaction.
7. **External Orchestration:** Asynchronous background tasks are triggered (e.g., AI generation, email dispatch) if needed.
8. **Data Serialization:** Pydantic schemas serialize the output data, removing sensitive credentials before returning a JSON response.

Figure 10.1: Nexus PM API Request Processing Sequence

10.6 Validation Strategy

The API validation strategy uses Pydantic schemas to ensure data integrity:

- **Request Schema Validation:** Raw JSON payloads are validated against Pydantic schemas immediately upon receipt. This enforces type compliance, value constraints (e.g., minimum string lengths), and attribute structures before logic execution.
- **Output Response Models:** Response models serialize output objects, ensuring that sensitive data (e.g., hashed passwords, Google IDs) is excluded from the returned payload.
- **Pydantic Exception Mapping:** If validation fails, Pydantic raises a validation error. FastAPI intercepts this error and returns a structured JSON list of validation details with an HTTP 422 status code, helping developers debug input issues.

10.7 Error Handling

The API implements a consistent error handling strategy. Standard HTTP status codes are mapped to specific system outcomes, as outlined in Table 10.2.

Status Code	System Outcome	Architectural Context
`200 OK`	Success (Read/Update)	Request completed and returned success payload.
`201 Created`	Success (Write)	Resource created and committed successfully.
`204 No Content`	Success (Delete)	Resource deleted; no body returned.
`400 Bad Request`	Invalid Operations	Logical constraints violated (e.g., email already registered).
`401 Unauthorized`	Session Invalidation	Token signature missing, expired, or invalid.
`403 Forbidden`	Permission Violation	User lacks the required workspace role.
`404 Not Found`	Missing Entity	Target resource (project, task, comment) does not exist.
`422 Unprocessable`	Schema Invalidation	Request JSON failed Pydantic type validation checks.
`429 Too Many Requests`	Rate Limiting Block	Request count exceeded active limits in Upstash Redis.
`500 Internal Error`	Unexpected Exception	Database failure or unexpected error in the backend service.

Table 10.2: REST API HTTP Status Code Mapping

Custom exceptions (such as `AIServiceError` or database connection timeouts) are caught by global exception handlers, preventing internal stack traces from leaking to clients.

10.8 API Security

The API implements several security controls at the gateway layer:

- **CORS Access Whitelists:** Cross-Origin Resource Sharing (CORS) rules restrict access to authenticated frontend origins (e.g., the production Vercel domain), blocking unauthorized browser requests.
- **Pre-Signed URL Restrictions:** File uploads are restricted by validating file extensions and sizes before generating pre-signed Cloudflare R2 URLs.
- **Log Anonymization:** Sensitive user credentials and authorization cookies are redacted from server logs. Each request is logged alongside its `X-Request-ID` to support end-to-end tracing.
- **Rate Limiting:** To protect API resources, rate-limits are tracked in Upstash Redis and evaluated per IP address and authenticated user ID.

10.9 Versioning and Maintainability

The API is designed to support future updates and maintain backward compatibility:

- **Router Modularity:** Using FastAPI sub-routers organizes endpoints into logical modules, allowing developers to add new features without modifying existing route files.
- **Dependency Injection Pattern:** Injected dependencies (e.g., database sessions, user authentication, access control checks) simplify testing by allowing mock dependencies in unit tests.
- **API Versioning Strategy:** Future API changes will use URL path versioning (e.g., `/api/v1/...`, `/api/v2/...`), letting clients upgrade to new API versions at their own pace.

10.10 External Integrations

The backend API coordinates transactions with several external managed services:

- **Supabase (PostgreSQL):** Acts as the primary database connection target, using PgBouncer for transaction-level connection pooling.
- **Cloudflare R2:** Provides S3-compatible cloud storage. The API generates short-lived pre-signed URLs, letting clients upload assets directly to R2.
- **Upstash Redis:** Stores API rate-limiting logs, session blocks, and cached dashboard data.
- **Resend:** Receives email JSON structures from background tasks and dispatches invitations and update notifications.
- **Google Gemini AI:** Receives task contexts and returns structured JSON lists for task suggestions.

10.11 Chapter Summary

This chapter detailed the API design for the Nexus PM gateway. By establishing stateless REST endpoints, utilizing Pydantic for validation, and structuring request pipelines, the API ensures high performance and security. The consistent handling of errors, authentication, and external integrations supports the platform's reliability. The next chapter will build on this design, detailing the deployment architecture and CI/CD pipelines.

CHAPTER 11

Security Design

11.1 Introduction

Security is a primary quality attribute of modern multi-tenant Software-as-a-Service (SaaS) environments. For collaboration platforms like Nexus PM, maintaining data confidentiality, tenant isolation, and identity validation is critical to protecting user workspaces and sensitive project metadata.

The purpose of this chapter is to detail the security design of Nexus PM. It outlines the platform's security philosophy, access token lifecycles, role-based authorization matrices, data encryption standards, and threat mitigation models.

11.2 Security Architecture

The security architecture of Nexus PM is built on four core design philosophies:

- **Defense in Depth:** Rather than relying on a single security boundary, the platform enforces validation checks at multiple layers, including transport encryption (TLS), gateway rate limiting, stateless JWT validation, service-layer role checks, and database-level relational integrity.
- **Least Privilege:** Users and service components are granted only the permissions required to execute their specific functions. Project roles restrict actions to those matching the user's role (e.g., Viewers cannot edit tasks).
- **Secure by Default:** Security controls are active by default. Authentication is required for all API endpoints unless explicitly flagged as public (e.g., login, register). Cookies default to maximum security settings.
- **Separation of Responsibilities:** Logical boundaries isolate core systems. File assets bypass the API compute threads, running directly to Cloudflare R2, and external integrations (e.g., Gemini AI, Resend) are managed separately from core database transactions.

11.3 Authentication

Authentication verifies user identities, supporting both local credentials and federated Google OAuth logins.

11.3.1 Stateless JSON Web Tokens

Upon successful authentication, the platform issues an asymmetric access token and a refresh token:

- **Access Token:** A short-lived JWT (valid for 15 minutes) signed with HMAC-SHA256. It contains the user identity (`sub`), scope details, and a unique JWT ID (`jti`) to prevent replay attacks. Access tokens are validated statelessly by the API gateway.
- **Refresh Token:** A long-lived JWT (valid for 7 days) signed with a separate key. Unlike access tokens, refresh tokens must be verified against a database table to support manual session revocation and automated token rotation.

11.3.2 Refresh Token Hashing

To prevent database compromise from exposing active user sessions, refresh tokens are hashed using SHA-256 (`hash_token`) before being stored in the database. When a client requests a new access token, the incoming refresh token is hashed and compared against the database records, protecting active sessions in the event of database dumps.

11.3.3 OTP Verification Security

For local user registration and password recovery workflows, the system enforces secure one-time password (OTP) verification:

- **Cryptographic Hashing:** OTP codes are generated as random 6-digit integers, which are then hashed using SHA-256 before database insertion in the `password_reset_otps` table, protecting OTP credentials in the event of a database compromise.
- **Strict Expiration Limits:** OTP codes expire 15 minutes after generation. An automated background job running in the FastAPI lifecycle (`cleanup_expired_otps_job`) runs every 24 hours to delete expired OTP records.
- **Rate-Limit Protections:** Brute-force guessing attacks on OTP validation are blocked by logging and checking request rates on the ‘/verify-otp‘ and ‘/forgot-password‘ routes using the database-backed `forgot_password_rate_limits` log table.

11.3.4 Cookie Security Specifications

Tokens are returned to the client using secure cookies configured with strict attributes:

- `HttpOnly` prevents client-side JavaScript from accessing cookies, mitigating Cross-Site Scripting (XSS) risks.
- `Secure` restricts cookie transmission to HTTPS channels, preventing token exposure over unencrypted networks.
- `SameSite` settings default to `Lax` (or configured via environment variables) to secure cookies on cross-origin requests, mitigating Cross-Site Request Forgery (CSRF) vulnerabilities.

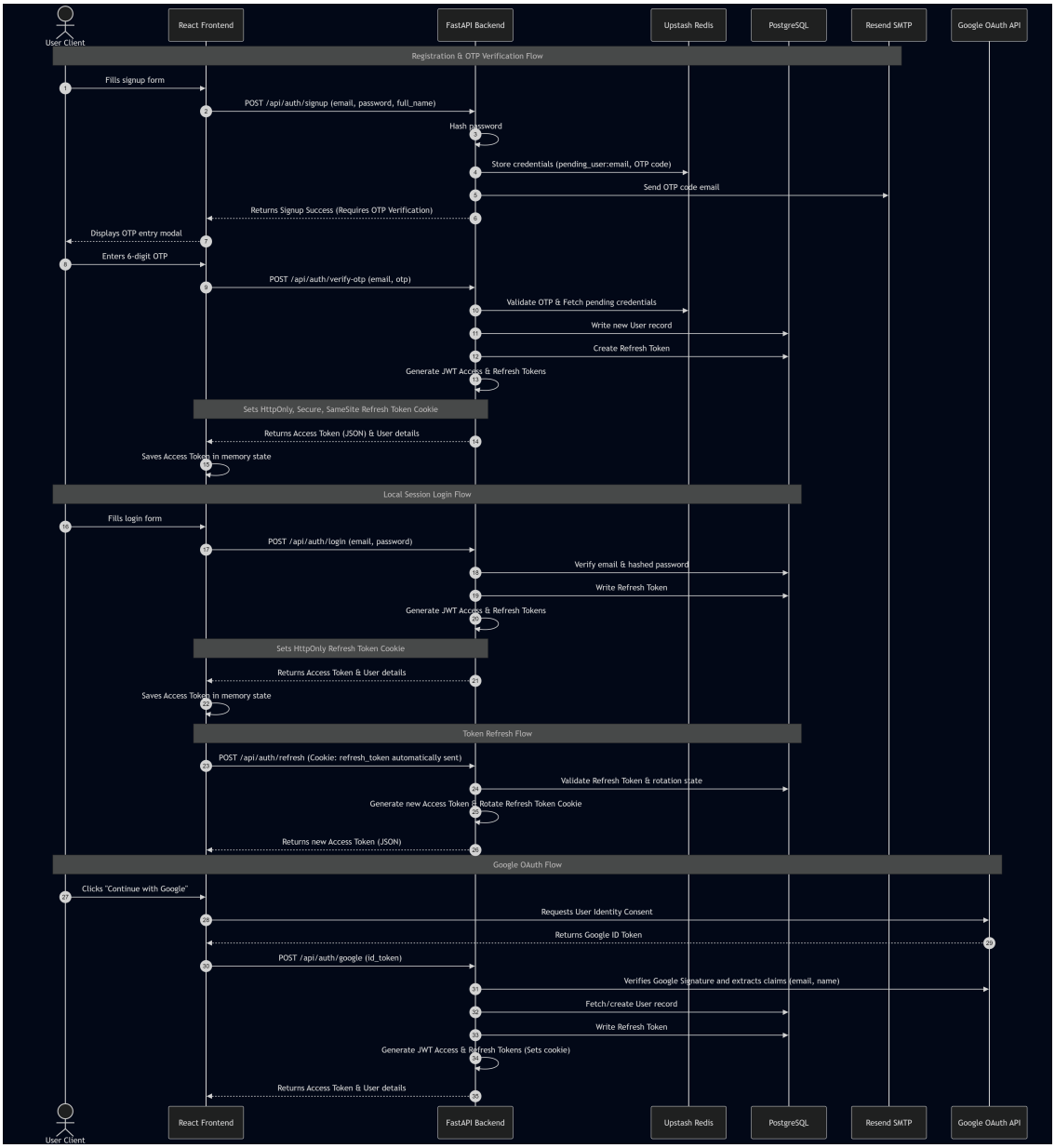

Figure 11.1: Nexus PM User Authentication and Token Verification Flow

11.4 Authorization

Authorization enforces data access boundaries and role-based permissions across workspaces and projects.

11.4.1 Role-Based Access Control (RBAC)

Access control is managed using project-level roles. A user's permissions within a project are determined by their role mapping in the `project_members` table, as outlined in the authorization matrix in Table 11.1.

Project Role	View Tasks	Create Tasks	Edit Tasks	Delete Tasks	Manage Members	Delete Project
Owner	Yes	Yes	Yes	Yes	Yes	Yes
Manager	Yes	Yes	Yes	Yes	Yes	No
Developer	Yes	Yes	Yes	No	No	No
Viewer	Yes	No	No	No	No	No

Table 11.1: Role-Based Access Control (RBAC) Permission Matrix

11.4.2 Access Control Verification

Permissions are validated dynamically before executing router handlers:

1. The gateway parses the request to identify the target project ID.
2. The service layer queries the `project_members` table to verify that the user is an active member.
3. The dependency injection handler evaluates the user's role against the endpoint's permitted roles.
4. If validation fails, an HTTP 403 Forbidden exception is raised, terminating the request.

11.5 API Security

Gateway security controls validate incoming requests and sanitize outgoing responses:

- **Input Validation:** Request bodies are parsed against Pydantic schemas, enforcing type constraints and length limits before data reaches service logic.
- **Output Serialization:** Response models filter sensitive database columns (e.g., password hashes, Google IDs), preventing accidental data leaks.
- **Error Sanitation:** Global exception handlers capture raw database and service errors, returning generic error codes to clients while writing detailed diagnostic traces to secure server logs.

11.6 Data Security

Data security controls protect information at rest and in transit:

- **Credential Hashing:** Local user passwords are encrypted using the bcrypt key-derivation function. The system uses a work factor of 12 rounds to resist brute-force attacks.
- **Secrets Management:** Private API keys, database URLs, and JWT keys are stored as system environment variables and loaded at runtime, preventing hardcoded credentials in code repositories.
- **Transport Layer Encryption:** All network traffic is encrypted using TLS 1.3, protecting session cookies and API tokens from interception.

11.7 File Storage Security

The storage module manages file attachments by coordinating direct uploads to Cloudflare R2, keeping large file streams off the API backend:

- **Pre-Signed URL Constraints:** Upload authorization is restricted by generating pre-signed URLs with a 15-minute expiration window.
- **Size and Type Validation:** Before generating upload URLs, the backend validates that file sizes are under limits (e.g., 10MB) and MIME types match a whitelist (e.g., PDF, PNG, JPEG), blocking execution of malicious scripts.
- **Direct Upload Bypass:** Direct uploads to R2 prevent backend thread blocks, reducing vulnerability to denial-of-service (DoS) resource exhaustion attacks.

11.8 External Service Security

Integrating external managed services requires secure communication boundaries:

- **Supabase (PostgreSQL):** Connections use secure SSL/TLS, and connection pooling (PgBouncer) transaction modes are utilized to prevent connection exhaustion.
- **Cloudflare R2:** Requests use AWS Signature Version 4 protocols, validating authorization keys on every API interaction.
- **Upstash Redis:** Access requires password authentication over TLS, securing rate-limiting and session cache operations.
- **Resend & Gemini AI:** Communication with email and LLM gateways runs over secure HTTPS API endpoints using validated API keys.

11.9 Threat Analysis

The Nexus PM design implements mitigations to address common security threats, as detailed in Table 11.2.

Threat Name	System Impact	Nexus PM Mitigation
Unauthorized API Access	Leakage of workspace data	Stateless JWT verification enforced across all endpoints.
Privilege Escalation	Unauthorized task modifications	Route permissions checked using dependency injection.
Token Theft (XSS/CSRF)	Account takeover via hijacked tokens	Access tokens isolated, and refresh tokens secured in HttpOnly cookies.
SQL Injection	Database data exposure or theft	SQL parameterization enforced using the SQLAlchemy ORM.
File Upload Abuse	Script execution in browser contexts	Size validation and MIME-type checks verified before upload.
Rate Abuse	Denial-of-Service (DoS) events	API rate limits tracked in Upstash Redis.
Credential Compromise	Session takeover via database dump	Password hashing using bcrypt and refresh token hashing using SHA-256.

Table 11.2: Threat Analysis and Architectural Mitigations Matrix

11.10 Security Best Practices

The platform incorporates secure software engineering practices:

- **Dependency Vulnerability Checks:** Container builds use multi-stage Docker configurations to reduce the attack surface, and dependency vulnerability scans (e.g., Snyk or Dependabot) are run regularly.
- **X-Request-ID Tracking:** Every API interaction is logged with a trace identifier, supporting security auditing and diagnostics without logging sensitive user data.

11.11 Future Security Enhancements

Future iterations of the platform plan to incorporate several security enhancements:

- **Multi-Factor Authentication (MFA):** Implement Time-based One-Time Passwords (TOTP) to secure local password logins.
- **HTTP Security Headers:** Configure Content Security Policies (CSP), HTTP Strict Transport Security (HSTS), and framing protections (`X-Frame-Options`) at the frontend hosting layer.
- **Automated Secrets Rotation:** Implement automated secret keys and database credential rotation workflows using cloud vault managers.

11.12 Chapter Summary

This chapter detailed the security design of the Nexus PM platform, specifying its authentication standards, role-based authorization matrices, API validations, and external integration protections. By using secure cookies, hashing credentials, and validating file uploads, the architecture protects sensitive project metadata and ensures tenant isolation. The next chapter will focus on the testing strategies used to verify these security controls and system functionality.

CHAPTER 12

AI Design

12.1 Introduction

Artificial Intelligence has transitioned from an experimental capability to a core component of modern productivity applications. In project management platforms, the integration of generative AI serves as a productivity multiplier rather than an autonomous replacement for human oversight. The role of AI in Nexus PM is to reduce the administrative overhead of task decomposition, metadata drafting, and project reporting.

By automating repetitive activities (such as breaking down project briefs into actionable checklists or compiling weekly status updates from task timelines), Nexus PM allows software engineering teams, product managers, and developers to focus on core execution. This chapter outlines the architecture, data flows, validation strategies, performance characteristics, and security parameters that define the AI subsystem.

12.2 AI Architecture

The AI subsystem uses a decoupled processing pipeline, sending requests asynchronously to external Large Language Models (LLMs) to prevent blocking main execution threads. Figure 12.1 illustrates the system context and runtime boundary configurations.

The lifecycle of an AI request flows through several components:

1. **Client Ingress:** The user initiates an AI action (e.g., requesting task suggestions) on the React client.
2. **Context Aggregation:** The FastAPI backend receives the request, validates the project context, and queries the database for relevant metadata.
3. **Prompt Assembly:** The prompt builder compiles project variables into formatted prompt templates.
4. **LLM Execution:** The AI service dispatches the request to the Google Gemini 2.5 Flash API asynchronously over HTTPS.
5. **Response Validation:** The response parser extracts the text block, validates the JSON format against Pydantic schemas, and commits the structured resources to the PostgreSQL database.

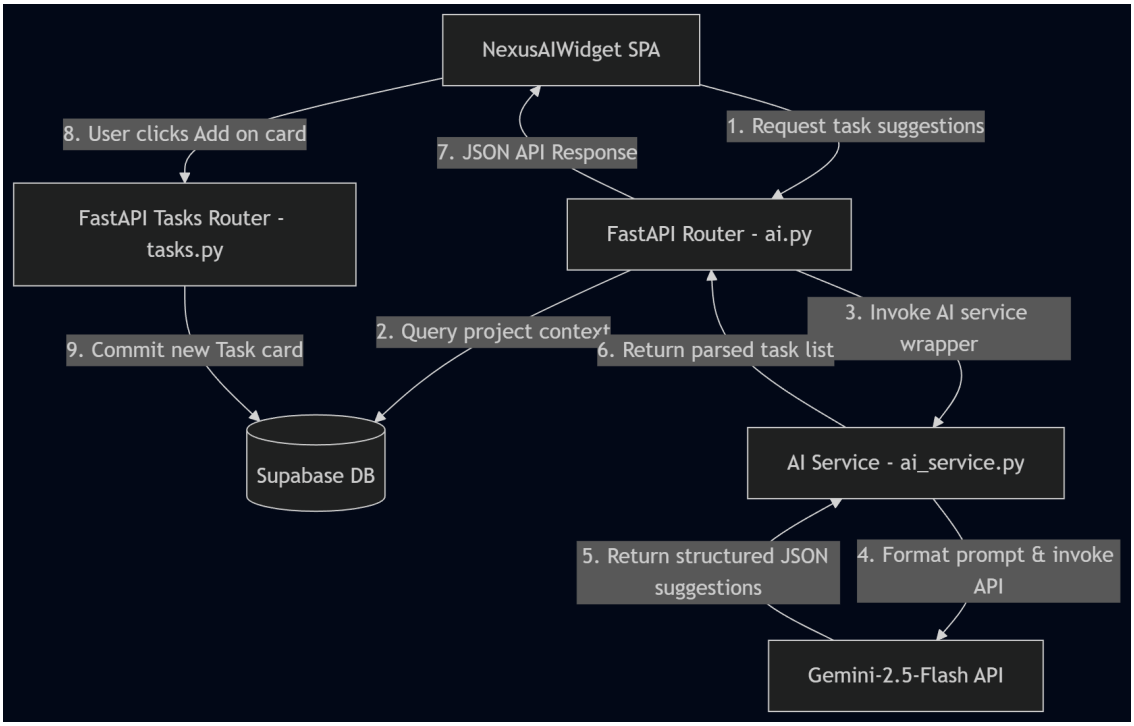

Figure 12.1: Nexus PM Asynchronous AI Subsystem and Data Flow Architecture

6. **State Synchronization:** The client application is notified of the completed generation, updating the interface state.

12.3 AI Components

The AI subsystem is composed of five logical components, summarized in Table 12.1.

AI Component	Subsystem Responsibility
GenAI Client	Manages the lifecycle of connection pools to Google's API gateway over HTTPS, using API keys from system configurations.
Prompt Builder	Compiles prompt templates, merges database variables, and sets system instructions.
Response Parser	Isolates JSON string blocks from LLM responses, running regex cleanups to strip markdown code blocks.
Validation Engine	Uses Pydantic models to validate structural type formats, preventing malformed data from being saved.
Error Handler	Maps API timeouts, rate-limit failures, and gateway errors to clean application exceptions.

Table 12.1: AI Subsystem Component Responsibilities

12.4 Current AI Features

The platform implements four primary AI capabilities, detailed in Table 12.2.

Feature Name	Backend Endpoint	Functional Workflow
Task Suggestions	`generate_tasks_from_description`	Reads project description text, generates 5–7 actionable task suggestions, and returns them as a structured JSON array.
Project Summaries	`summarize_project`	Aggregates project descriptions, category metadata, and active task statuses to generate a text summary.
Task Descriptions	`generate_task_description`	Accepts short task titles and context variables, generating detailed task descriptions.
Meeting Minutes to Tasks	`meeting_notes_to_tasks`	Parses raw meeting transcripts and notes, extracting actionable tasks and assigning estimated timelines.

Table 12.2: Implemented AI Capabilities

12.5 Prompt Engineering Strategy

To guarantee system stability, the platform uses a structured prompt engineering strategy:

- **Role-Based System Instructions:** System prompts define the model's role (e.g., "You are a project management assistant") and output constraints (e.g., requesting raw JSON formatting).
- **Context Minimization:** The Prompt Builder includes only essential project metadata in request templates, minimizing token consumption and reducing processing latencies.
- **Structured Output Constraints:** Prompts define expected JSON schema parameters, including required fields, string lengths, and element counts, to ensure response consistency.
- **Deterministic Tuning:** Model configuration parameters (such as temperature) are tuned to ensure consistent, deterministic outputs across requests.

12.6 Response Processing

Since generative language models can occasionally return malformed responses, the backend implements a structured response validation workflow:

1. **Markdown Cleanups:** The response parser strips markdown wrapper text (e.g., ```` ```json ... ``` ````) to isolate raw JSON strings.
2. **JSON Validation:** The cleaned response string is validated using Python's `json.loads`.
3. **Pydantic Schema Validation:** The JSON object is parsed against Pydantic schemas (e.g., `TaskSuggestion`), verifying type constraints and field structures.

4. **Sanitization Fallbacks:** If parsing or validation fails, the service logs the failure, attempts to extract partial fields, and falls back to a clean error response rather than committing malformed data.

12.7 Performance Considerations

Integrating generative APIs introduces latency and rate limit challenges, which the platform manages using several optimizations:

- **Asynchronous API Calls:** AI integrations use FastAPI's asynchronous support (`client.aio`), preventing API network latency from blocking the server's main request loop.
- **Rate-Limit Handling:** The service detects Gemini rate limits (HTTP 429), logging the event and returning user-friendly error messages to the client.
- **Cache Policies:** Analytical project summaries are cached in Upstash Redis, preventing redundant API calls when project data has not changed.

12.8 Security and Privacy

AI features are designed to protect user data and secure integration boundaries:

- **Input Sanitization:** Input context is sanitized using Pydantic validation schemas to mitigate prompt injection risks.
- **Access Authorization:** Endpoints enforce workspace role checks (`require_project_role`), ensuring users can only run AI actions on projects they are member of.
- **Credential Isolation:** Gemini API keys are loaded via system environment variables, preventing exposure in client code or code repositories.

12.9 Future AI Roadmap

Future iterations of the platform plan to expand AI capabilities, as outlined in the roadmap in Table 12.3.

12.10 Architectural Strengths

The AI integration architecture delivers several key advantages:

- **Structured Output Validation:** Validating AI responses against strict schemas ensures database integrity and prevents application parsing failures.
- **Low Operational Overhead:** Leveraging managed API services avoids the compute and maintenance costs of hosting local model engines.
- **Non-Blocking User Experience:** Asynchronous task execution ensures that the user interface remains active and responsive while generative processes run.

Roadmap Goal	Planned Capabilities
AI Sprint Planning	Automated velocity estimations, resource balancing recommendations, and milestone groupings based on historical sprint performance.
AI Risk Analysis	Automated identification of blocking task dependencies, project delays, and timeline bottleneck predictions.
AI Bug Prioritization	Classification of incoming bug reports based on stack traces, assigning priorities, and routing issues to specific developers.
Natural Language Search	Semantic workspace search capabilities, letting users query project status using conversational queries.

Table 12.3: Future AI Subsystem Development Roadmap

12.11 Chapter Summary

This chapter detailed the AI design of the Nexus PM platform, outlining the prompt engineering pipelines, validation strategies, performance configurations, and security boundaries. By using Google Gemini 2.5 Flash and enforcing structured JSON validation, the platform provides reliable AI capabilities while protecting workspace data. The next chapter will focus on the deployment architectures and operational topologies.

CHAPTER 13

Deployment Design

13.1 Introduction

The deployment architecture defines the physical hosting environment, network layout, continuous integration/continuous deployment (CI/CD) pipelines, and operational configurations for the Nexus PM platform. Designing a production environment requires balancing availability, data residency, latency, and resource constraints.

The deployment design for Nexus PM is built around containerized backend environments and edge-cached frontends, using managed cloud services to minimize operational maintenance. This chapter details the platform's multi-cloud infrastructure, deployment topologies, database connection rules, secure networks, environment variables, and deployment workflows.

13.2 Cloud Infrastructure Overview

Nexus PM uses a multi-cloud topology, deploying components to hosting providers based on their specific performance and billing characteristics. Table 13.1 summarizes the providers and their roles.

Provider	Service Component	Operational Focus
Vercel	Frontend Web Client	Global CDN distribution, edge caching, and automated client builds.
Render	Backend REST API Engine	Stateless Docker container hosting with health checks and logs.
Supabase	Managed PostgreSQL Database	ACID storage, automated backups, and PgBouncer connection pooling.
Cloudflare	R2 Object Storage	S3-compatible file storage with zero egress bandwidth charges.
Upstash	Serverless Redis Cache	Serverless in-memory cache registry, session cache, and rate-limiting store.
Resend	Email Gateway	API-driven transactional email dispatching.

Table 13.1: Nexus PM Cloud Hosting Providers

13.3 Deployment Topology

The runtime deployment topology is designed to secure communication and minimize latency between the client browser, API gateway, databases, and external cloud services, as shown in Figure 13.1.

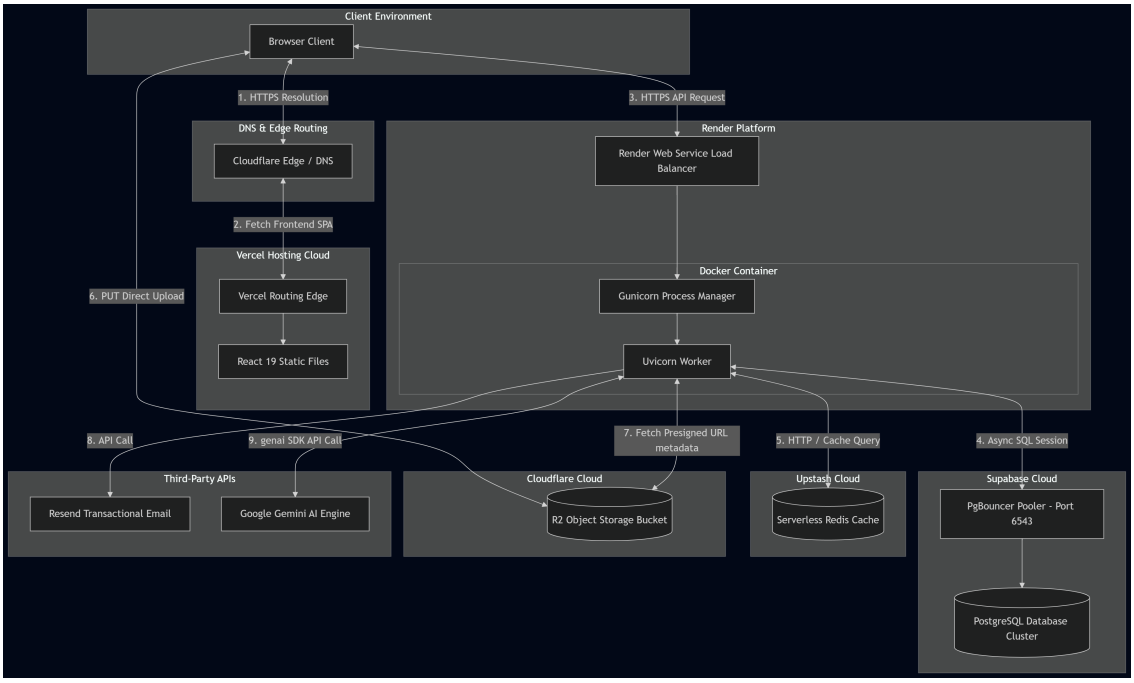

Figure 13.1: Nexus PM Production Deployment Topology and Network Flow

The division of hosting responsibilities between internal modules and external endpoints is outlined in Table 13.2.

Service Layer	Deployment Node	Interface Responsibility
Presentation Client	Client Web Browser	Loads React static bundles from Vercel edge routers.
API Controller	Render Docker Host	Decodes API requests, validates inputs, and coordinates services.
Relational Storage	Supabase Database Node	Handles transactional SQL operations using PgBouncer.
Cache Layer	Upstash Redis Node	Manages session states and rate limits.
Asset Storage	Cloudflare R2 Bucket	Handles binary file transfers using pre-signed upload URLs.
AI Reasoning	Google Gemini API Gateway	Processes prompt contexts asynchronously.
Outbound SMTP	Resend Dispatch Node	Sends transaction alerts and workspace invitations.

Table 13.2: Nexus PM Component Hosting Allocations

13.4 Frontend Deployment

The presentation layer is deployed to the **Vercel** platform as a compiled single-page application (SPA):

- **Build Pipeline:** On code commits to the main repository, Vercel pulls the code, resolves dependencies, and compiles the React 19 codebase using Vite.
- **Edge Distribution:** The compiled HTML, CSS, and JS bundles are cached globally across Vercel's edge nodes. This routes initial client requests to the nearest edge location to minimize latency.
- **Routing and Caching:** Vercel handles client-side routing, falling back to `index.html` for unresolved paths to let the client routing framework handle transitions. Static resources are cached with long-lived HTTP headers to reduce traffic.

13.5 Backend Deployment

The API backend is hosted on **Render** as a containerized web service:

- **Docker Runtime:** The FastAPI service runs inside a lightweight Docker image based on `python:3.12-slim`. System dependencies (e.g., `libpq-dev`) are installed during build time.
- **Startup Routine:** The container uses `entrypoint.sh` to run database migrations (`alembic upgrade head`) before starting the web server. If migrations fail, the startup crashes, preventing the container from serving traffic with outdated schemas.
- `Uvicorn` handles ASGI routing inside the container, managed by a Gunicorn master process configured with a single worker worker to operate within Render's free-tier memory limits.
- **Cold Start Behavior:** On Render's free tier, containers enter sleep mode after 15 minutes of inactivity. Subsequent requests trigger a container rebuild and startup routine, causing a 30-50 second delay for the initial request.

13.6 Database Deployment

The database layer uses PostgreSQL hosted on **Supabase**:

- **Connection Optimization:** Relational queries run over secure TCP connections. To manage database connection limits on the free tier, the API routes queries through a PgBouncer instance running in transaction-pooling mode.
- **Connection Pool Settings:** Because PgBouncer in transaction mode is incompatible with prepared statement caches, the SQLAlchemy database engine is configured with `"statement_cache_size": 0` to prevent query compilation errors.
- **Migrations and Backups:** Alembic upgrades run on container startup to keep the schema in sync. The database service schedules daily snapshot backups to support disaster recovery.

13.7 Object Storage

File attachments are stored in an S3-compatible bucket hosted on **Cloudflare R2**:

- **Zero Egress Fees:** R2 was chosen to eliminate egress bandwidth fees, ensuring predictable operational costs as file volume grows.
- **Direct Upload Workflow:** The API generates pre-signed upload URLs, letting clients upload files directly to R2. This bypasses the backend API containers, saving compute resources and reducing network bottleneck risks.
- **Access Authorization:** Attachments are secured using unique file keys, and the backend validates task membership permissions before generating download links.

13.8 Redis Cache

In-memory caching is hosted on **Upstash Redis**:

- **Rate Limiting:** API endpoints enforce rate limiting by tracking request counts per IP and user ID in Redis.
- **Serverless Deployment:** The serverless Redis cluster scales compute resources automatically based on traffic volume, avoiding the need to provision dedicated Redis nodes.
- **Session State Caching:** Redis caches active session tokens, checking revocations before validating access tokens.

13.9 External Services

Integrating external managed APIs keeps the application footprint low:

- **Google OAuth Gateway** delegates authentication security, eliminating local credential storage risks.
- **Resend API** handles transactional email dispatches, offloading SMTP server maintenance.
- **Google Gemini API** provides Large Language Model capabilities, processing task generation requests.

13.10 Networking

Network configuration focuses on securing data transit:

- **Domain Management:** The platform uses a custom domain (`nexuspm.online`). Vercel routes traffic, and Render secures the API gateway domain (`api.nexuspm.online`).
- **Transport Security:** All ingress and egress traffic must use HTTPS (TLS 1.3), preventing packet sniffing on public networks.
- **CORS Configuration:** Cross-Origin Resource Sharing (CORS) rules restrict access to the production frontend domain, blocking unauthorized browser requests.

13.11 Environment Configuration

Configuration settings are loaded dynamically from environment variables using Pydantic Settings classes. Table 13.3 outlines the key settings used to configure the platform.

Config Variable	Source	System Impact
`DATABASE_URL`	Supabase	Connection string for the PostgreSQL database, routed through PgBouncer.
`REDIS_URL`	Upstash	Connection URI for the Redis cache.
`SECRET_KEY`	Local Key	Cryptographic key for signing local access tokens.
`REFRESH_SECRET_KEY`	Local Key	Cryptographic key for signing refresh tokens.
`R2_BUCKET_NAME`	Cloudflare	Name of the Cloudflare R2 storage bucket.
`GEMINI_API_KEY`	Google	API key for authenticating Gemini AI requests.
`RESEND_API_KEY`	Resend	API key for authenticating transactional email requests.

Table 13.3: Key System Configuration Variables

13.12 Deployment Pipeline

The deployment pipeline uses automated Git integration to coordinate deployments:

1. **Git Commit:** Developers push code updates to the central GitHub repository.
2. **Frontend Build:** Vercel detects updates in the client codebase, compiles the static assets, and deploys the new version.
3. **Backend Build:** Render detects updates in the backend codebase, pulls the source code, builds the Docker image, and deploys the container.
4. **Migration Run:** The backend container runs the database migration script (`alembic upgrade head`) on startup, keeping the schema in sync before starting the API server.

13.13 Operational Considerations

Operating the platform requires managing the trade-offs of free-tier hosting limits:

- **Render Cold Starts:** Free-tier containers enter sleep mode after 15 minutes of inactivity. When a request is received, the container rebuilds and starts up, causing a 30-50 second delay. To keep the container active, automated ping services (e.g., cron jobs) trigger health checks every 10 minutes.
- **Logging and Diagnostics:** Backend logs are routed to Render's logging interface. The `X-Request-ID` trace header is appended to logs to support request tracing.
- **Disaster Recovery:** The managed database service schedules daily snapshot backups to support disaster recovery, while Cloudflare R2 replicates files across edge storage nodes.

13.14 Future Improvements

Future updates to the deployment architecture plan to address several scaling improvements:

- **Infrastructure as Code (IaC):** Define system resources using tools like Terraform to automate infrastructure setups.
- **Dedicated Compute Nodes:** Migrate from free-tier containers to dedicated instances (e.g., AWS ECS or Kubernetes) to eliminate cold start delays.
- **Automated Secrets Management:** Move environment secrets to a managed vault service (e.g., AWS Secrets Manager) to secure key configurations.

13.15 Chapter Summary

This chapter detailed the deployment design for the Nexus PM platform, specifying its multi-cloud hosting, container configurations, database connection rules, and deployment pipelines. By combining Vercel CDN static deliveries, containerized Render web services, and managed cloud resources, the architecture delivers a scalable, cost-efficient hosting environment. The next chapter will outline the testing protocols used to verify this configuration.

CHAPTER 14

Testing Strategy

14.1 Introduction

A systematic verification and quality assurance strategy is critical to maintaining software quality, operational reliability, and deploy confidence in a cloud-native, multi-tenant SaaS application. For Nexus PM, the testing strategy serves to verify functional correctness, validate security controls (e.g., workspace isolation and role permissions), and prevent regression errors during continuous integration and deployment.

This chapter details the testing objectives, testing levels, backend and frontend validation procedures, post-deployment verifications, security audits, and the automated CI/CD quality gates implemented in the GitHub Actions pipeline.

14.2 Testing Objectives

The quality assurance strategy of Nexus PM is designed around seven engineering objectives:

- **Functional Correctness:** Verify that REST endpoints return expected JSON payloads and data modifications are written to PostgreSQL in accordance with schema rules.
- **Reliability:** Ensure that the stateless API gateway handles load spikes, rate-limit thresholds, and database connection timeouts gracefully.
- **Security Validation:** Verify that authentication tokens are parsed statelessly, secure cookie flags are present, and workspace role boundaries prevent unauthorized access.
- **Performance Validation:** Track endpoint response latencies, database execution times, and external API call runtimes.
- **Cloud Deployment Validation:** Ensure that container builds compile without error and database schema migrations execute successfully.
- **Regression Prevention:** Automate test runs in CI pipelines to catch software regressions before code gets merged to active branches.

- **Code Maintainability:** Encourage modular, clean coding patterns by enforcing strict linting, formatting, and dependency audits.

14.3 Testing Levels

The verification framework of Nexus PM is split across five distinct testing levels, categorizing implemented vs. future procedures in Table 14.1.

Testing Level	Implementation Status	Verification Methodology
Static Analysis	Active (CI Pipeline)	Formatting verification using Black, code linting via Ruff, and package security audits.
Unit Testing	Active (Local & CI)	Database schema validations and attachment logic testing using Python's `unittest` library.
Integration Testing	Future Work	Verification of API routes, Redis caching actions, and OAuth flows using Pytest-asyncio.
System Testing	Manual (Verification)	End-to-end workspace setup and task board interaction tests conducted in staging containers.
E2E Browser Validation	Future Work	Automated user interactions and view state updates verification using Playwright or Cypress.

Table 14.1: Nexus PM Testing Levels and Status Specifications

14.4 Backend Validation

Backend validation checks model relationships, service logic, and query executions:

- **Schema and Relation Validation:** The backend test suite (managed via Python's standard `unittest` library) verifies database constraints. For example, `tests/test_task_attachments.py` validates that task attachment relationships, cascade deletions, and user-deletion bindings conform to database definitions.
- **Route Import and Syntax Auditing:** To prevent execution errors on deployment nodes, the test runner compiles the python source files (`compileall`) and imports the main router class (`import main`) in a test environment, checking syntax and dependency imports before executing backend tests.
- **Transaction Verification:** Scoped database session lifecycles are checked to ensure rollback execution on exceptions, preventing data corruptions.

14.5 Frontend Validation

Because automated user interface tests are not yet implemented, frontend verification combines static builds compilation checks with manual review:

- **Compilation Checks:** The frontend pipeline executes static bundle compilation runs (`npm run build`). This checks that the TypeScript parser catches syntax errors, interface mismatches, or invalid imports before deployment.
- **Static Code Checks:** Code formatting and style rules are checked using ESLint rules (`npm run lint`) to ensure clean syntax.
- **Manual UI Audits:** Key interface elements (e.g., responsive page layouts, Kanban drag-and-drop actions, JWT refresh loops, Google sign-in prompts) are manually verified in development and staging environments.

14.6 Deployment Validation

Following deployment to Render and Vercel, post-deployment verifications check that production resources are configured correctly:

- **Connection Status:** Verification checks confirm that the backend container can communicate with the Supabase database instance and Upstash Redis caches over TLS.
- **Domain and SSL Audits:** DNS redirects and custom domain bindings (`nexuspm.online`) are audited to ensure proper routing and SSL/TLS transport encryption.
- **Config Settings Validation:** Build processes check that environment variables (e.g., API keys, database credentials) are set correctly in production containers, avoiding missing variable errors during runtime.

14.7 Security Verification

Security audits verify that authorization controls are active and data access is restricted:

- **JWT Signature Validation:** API controllers verify access tokens, checking signature keys and expirations, and raising unauthorized errors on invalid tokens.
- **Cookie Settings Verification:** Browser inspectors verify that session refresh cookies contain the `HttpOnly`, `Secure`, and `SameSite=Strict` flags in production.
- **Role Permissions Verification:** Request handlers test authorization boundaries, checking that restricted resources (e.g., deleting projects) return 403 Forbidden errors when accessed by unauthorized roles.

14.8 Performance Validation

Performance audits monitor resource limits and response times under standard loads:

- **API Latency Tracking:** Read and write latencies are monitored to ensure requests resolve within performance targets (e.g., under 100ms for read paths).
- **Database Query Optimizations:** Database queries are checked using PostgreSQL execution plan analysis (`EXPLAIN ANALYZE`) to ensure indexes are utilized and tables are scanned efficiently.
- **Cold Start Auditing:** Compute warm-up routines are monitored to check container start times from dormant states on Render's free tier.

14.9 CI/CD Quality Gates

Nexus PM automates code validation using GitHub Actions. The pipeline defined in `.github/workflows/ci.yml` enforces several build quality gates before code changes can be merged:

1. **Security Auditing (Black/Ruff/pip-audit):** Python files are checked for formatting compliance (`black --check .`) and linting errors (`ruff check .`). Active dependencies are scanned for vulnerability exploits (`pip-audit -r requirements.txt`).
2. **Frontend Auditing (npm audit/lint/build):** Packages inside the `frontend` directory are audited for security risks. Build pipelines verify that the React client compiles without errors.
3. **Migration Run Verification (database-ci):** A local PostgreSQL instance is spun up via Docker service containers. The runner executes Alembic migrations (`alembic upgrade head`) to verify database changes and migration head status.
4. **Unit Tests Executions (backend-ci):** Python imports are checked, and the unit test suite is executed using the standard test runner (`python -m unittest discover`).
5. **Docker Compilation Checks (docker-ci):** The pipeline builds Docker containers for both backend and frontend environments, ensuring the images compile correctly before deployment.

14.10 Risks and Testing Limitations

The testing strategy has several known limitations, which are managed as part of operational risk planning:

- **UI Testing Gaps:** The lack of automated frontend testing (such as component unit tests or end-to-end browser tests) increases the risk of interface regressions during updates. This risk is mitigated through manual testing workflows.
- **Load and Chaos Testing:** The platform has not undergone automated load, stress, or chaos testing. The lack of load testing makes it difficult to predict database behavior under high concurrent user loads.
- **AI Reliability Variance:** Generative outputs from Google Gemini can vary between requests. The non-deterministic nature of AI generations makes them difficult to test with simple assert checks, requiring input schema constraints.

14.11 Future Testing Roadmap

Future iterations of the platform plan to address several verification improvements, as outlined in the testing roadmap in Table 14.2.

Planned Goal	Verification Objectives
Pytest Migration	Migrate backend test suites from `unittest` to `pytest` to support async testing patterns and parameterized test runs.
Playwright Integration	Implement browser automation tests to verify Kanban drag-and-drop actions, login redirections, and client-side page routing.
OWASP Security Auditing	Automate security scans (e.g., OWASP ZAP) to check for common vulnerabilities like SQL injection or cross-site scripting.
Load and Performance Testing	Implement automated performance tests (e.g., k6) to benchmark API endpoints and database connection pools under simulated traffic.

Table 14.2: Future Quality Assurance Roadmap

14.12 Chapter Summary

This chapter detailed the testing strategy and quality assurance procedures for Nexus PM. By combining static code analysis, unit testing suites, and automated CI/CD quality gates in GitHub Actions, the architecture verifies code changes and database migrations before deployment. Managing testing limitations and executing post-deployment checks helps ensure system reliability and security. The next chapter will outline the operational readiness checklists and production configurations.

CHAPTER 15

Production Readiness Assessment

15.1 Introduction

Before a software application transitions from active development to full-scale production deployment, it must undergo a rigorous evaluation of its operational readiness. A Production Readiness Assessment (PRA) acts as an independent architectural audit, checking the platform's stability, security controls, system performance, deployment reliability, and recovery procedures.

For Nexus PM, a cloud-native, AI-powered project management platform, the PRA evaluates the operational state of the production environment. This assessment identifies system strengths, highlights known infrastructure limitations, maps potential risks, and provides a prioritized roadmap for scaling resources to meet production demands.

15.2 Architecture Readiness

The logical architecture of Nexus PM is evaluated as follows:

- **Decoupled Client-Server Separation:** The separation of the React client and the FastAPI backend router is a core strength. This design allows for independent scaling, testing, and deployment of presentation and business logic layers.
- **Modularity and Separation of Concerns:** The backend codebase enforces clean boundaries, separating route controllers, service handlers, database models, and repository interfaces. This structure ensures that updates to one component (e.g., database schemas or email templates) do not leak into unrelated layers.
- **Maintainability and Extensibility:** Strong static typing (TypeScript) on the client, combined with declarative validation schemas (Pydantic) on the server, ensures codebase maintainability. Adding new endpoints or integrating additional third-party APIs requires minimal adjustments to the core framework.

15.3 Infrastructure Readiness

The cloud infrastructure stack consists of managed SaaS and PaaS nodes:

- **Frontend Hosting (Vercel):** Ready for production. The edge CDN caching model ensures fast asset delivery and low latency for client requests globally.
- **Backend Compute (Render):** Capable of handling baseline API traffic within containerized environments. However, free-tier container sleep cycles introduce cold start delays, which must be addressed before production launch.
- **Relational Database (Supabase):** Capable of handling transactional operations. The configuration uses PgBouncer to manage connection limits, and automated daily backups are configured.
- **Asset Storage (Cloudflare R2):** S3-compatible, zero-egress object storage handles file attachments, removing storage and bandwidth costs.
- **Cache and Session Registry (Upstash):** Serverless Redis instances scale dynamically based on request volumes, handling rate-limiting and metadata caching.
- **Transactional Communications (Resend):** Handles automated invite dispatches and task assignment alerts.

15.4 Security Readiness

Security audits confirm that core protection mechanisms are active:

- **Stateless Access Control:** Session tokens are signed using HMAC-SHA256, and refresh tokens are rotated dynamically, protecting API routes.
- **Hashed Credentials:** User passwords are encrypted using bcrypt, and refresh tokens are hashed using SHA-256 before database storage to prevent session hijacking in the event of database leaks.
- **Cookie Security Flags:** Auth refresh tokens are stored in cookies flagged with `HttpOnly`, `Secure`, and `SameSite=Strict` in production.
- **Input Validation boundaries:** Incoming REST payloads are validated against Pydantic schemas, and file attachments are checked for size limits and MIME types.

15.5 Reliability & Availability

The system architecture includes several design choices to ensure reliability:

- **Fault Isolation:** Downstream service dependencies are decoupled from core database transactions. Failure of secondary APIs (such as email delivery or AI summaries) does not block core task management functions.
- **Compute Cold Starts:** Render's free tier container sleep cycle remains a primary availability bottleneck. Health-check warm-up pings are configured to keep instances active, but dedicated hosting is required for production scaling.
- **Recovery Considerations:** Supabase handles automated database backups, while Cloudflare R2 replicates files across edge locations to ensure data safety.

15.6 Performance Readiness

System metrics are optimized using several caching and transaction controls:

- **Endpoint Responsiveness:** Read operations are accelerated by caching aggregated dashboard metrics in Upstash Redis.
- **Database Connection Management:** PgBouncer connection pooling prevents connection exhaust issues under high API concurrent loads.
- **Non-Blocking Operations:** Direct file uploads to Cloudflare R2 and asynchronous LLM generations prevent backend thread blocking, maintaining fast response times.

15.7 Operational Readiness

Operations management enforces repeatability and consistency across deployments:

- **Deployment Repeatability:** Deployments use automated Git integrations, compiling frontend resources and building Docker containers on git pushes.
- **Startup Migrations:** The backend container runs database migrations automatically on startup via `entrypoint.sh`.
- `X-Request-ID` trace headers are appended to server logs, supporting request diagnostics without logging sensitive user data.

15.8 Known Limitations

The current production environment has several resource and monitoring limits:

- **Render Free-Tier Cold Starts:** Free-tier containers enter sleep mode after 15 minutes of inactivity, causing a 30-50 second delay on subsequent initial requests.
- **Limited Observability:** Storing logs locally on Render and Vercel consoles makes it difficult to trace issues across services without a centralized APM or log aggregation dashboard.
- **Single-Region Datacenter Lock:** Database and cache nodes run in a single primary region, leaving the system vulnerable to regional cloud provider outages.
- **Free-Tier API Limits:** Third-party integrations (Google Gemini, Resend) are restricted by free-tier API usage limits, which can block updates when quotas are exceeded.

15.9 Risk Assessment

Table 15.1 lists potential production risks along with their likelihood, impact, and mitigation strategies.

Identified Risk	Likelihood	Impact	Architectural Mitigation
Render container sleep	High	Medium	Health-check cron warm-up pings keep containers active.
Centralized DB outage	Low	Critical	Managed daily backups on Supabase for disaster recovery.
AI API quota exhaust	Medium	Low	Handle API errors gracefully and return generic user alerts.
Session token hijack	Low	Critical	Secure cookies (`HttpOnly`) and SHA-256 refresh token hashes in the DB.
APM monitoring gap	High	Medium	Custom log tracing using request IDs in ContextVars.

Table 15.1: Production Risk Assessment Matrix

15.10 Production Readiness Score

Table 15.2 provides a scorecard of the platform's production readiness across core categories.

Category	Score (1-10)	Architectural Justification
Architecture	9/10	Complete separation of concerns, modular backend layout, and stateless API gateway design.
Security	9/10	Secure cookie configurations, password hashing, and token encryption are active.
Deployment	8/10	Repeatable container builds and automated git deployment integrations are configured.
Performance	7/10	Client-side data caching is implemented. Cache speeds are fast, but Render free-tier latency remains a bottleneck.
Scalability	7/10	The API tier is stateless, but scaling is limited by database connection limits on the free tier.
Documentation	9/10	System manuals and installation guides are documented in the SAD repository.
Maintainability	9/10	Codebase is structured with clear layers, type definitions, and ORM abstractions.
Testing	7/10	Unit test coverage is active for DB schemas, but automated end-to-end user tests are missing.
Overall Score	**8.1/10**	**The platform is architecture-ready and secure, but requires dedicated hosting to resolve performance and scaling limits.**

Table 15.2: Production Readiness Scorecard

15.11 Recommendations

To transition Nexus PM to a fully production-ready environment, the following actions are recommended:

15.11.1 High Priority

- **Dedicated Hosting instance:** Upgrade Render containers to paid tiers to eliminate container sleep cycles and cold start delays.
- **APM Monitoring Tools:** Integrate a centralized log aggregator (e.g., Datadog or Sentry) to trace errors and monitor API latencies across Vercel, Render, and Supabase.

15.11.2 Medium Priority

- **Async Test Suite Migration:** Migrate the test suite from `unittest` to `pytest-asyncio` to improve validation of async routes.
- **Database Connection Upgrades:** Upgrade database connection limits and pool sizes as user traffic scales.

15.11.3 Low Priority

- **Infrastructure as Code (IaC):** Manage infrastructure configurations using Terraform scripts.
- **Multi-Region Database Replication:** Configure read-replica database nodes in target geographic zones to improve read times.

15.12 Chapter Summary

This chapter evaluated the production readiness of the Nexus PM platform, assessing its software architecture, deployment infrastructure, security controls, perform limits, and operational repeatability. While the core architecture and security controls are production-ready, free-tier hosting limits on Render introduce operational risks. Upgrading compute instances and adding centralized monitoring tools will resolve these limits, ensuring a reliable, enterprise-grade SaaS deployment.

CHAPTER 16

Future Roadmap & Architectural Evolution

16.1 Introduction

Software architecture is not a static blueprint but an evolving framework that must adapt to changing scale, user demands, and technological advancements. As a platform grows, its initial architectural assumptions must be re-evaluated to prevent technical debt, ensure high availability, and support developer velocity.

For Nexus PM, the current decoupled model and modular backend layout provide a solid foundation for initial deployment. However, scaling the platform to support enterprise workloads requires a structured architectural roadmap. This chapter defines the guiding principles, short-term optimizations, medium-term enhancements, and long-term architectural visions that will guide the evolution of the platform.

16.2 Guiding Architectural Principles

The architectural evolution of Nexus PM is guided by six engineering principles:

- **Maintainability:** Enforce clean boundaries between application modules, ensuring components remain decoupled and testable.
- **Horizontal Scalability:** Design components to scale horizontally, allowing compute resources to expand independently of database limits.
- **Secure by Design:** Continually upgrade security controls, implementing multi-factor authentication, security headers, and automated credential rotation.
- **Developer Experience (DX):** Enforce standard development containers, database seeding utilities, and automated CI pipelines to maintain high developer velocity.
- **Cloud-Native Optimization:** Maximize the use of serverless, zero-maintenance cloud components to minimize operational overhead.
- **Context-Aware Intelligence:** Integrate Large Language Models natively into standard workflows, focusing on productivity automation rather than external chat integrations.

16.3 Short-Term Roadmap (3–6 Months)

The short-term roadmap focuses on optimizing the existing codebase, expanding test coverage, and refining current AI features:

- **Vite Build Optimizations:** Implement code-splitting and lazy-loading in the React client to minimize initial bundle size and speed up page load times.
- **Test Suite Migration:** Migrate the test runner from standard Python `unittest` to `pytest-asyncio` to improve test execution speed and simplify async route mocking.
- **AI Feature Refinements:** Optimize prompt configurations for the task description generator and fine-tune response parsing logic for the meeting minutes parser.
- **Developer Tooling:** Create automated database seeding scripts and integrate OpenAPI schema checks in CI build pipelines.

16.4 Medium-Term Roadmap (6–12 Months)

The medium-term roadmap introduces real-time synchronization, dedicated background job queues, and advanced search capabilities:

- **WebSocket Integration:** Implement WebSocket endpoints in FastAPI, coordinated via Redis Pub/Sub, to sync Kanban board updates instantly across active collaborator views.
- **Asynchronous Task Queues:** Deploy a dedicated task manager (e.g., Celery or RQ) to handle long-running processes (AI generation, email alerts, file validation) outside the HTTP loop.
- **Full-Text Search:** Configure PostgreSQL full-text search indexes to allow fast, scoped searches across projects, tasks, and comments.
- **Centralized Observability:** Set up staging APM monitoring dashboards (e.g., Sentry or Prometheus) to trace errors and monitor endpoint latencies.

16.5 Long-Term Vision (1–3 Years)

The long-term vision prepares the platform for enterprise workloads, focusing on infrastructure automation and service isolation:

- **Microservices Refactoring:** If compute demands justify it, split the modular monolith into isolated microservices (e.g., separating the AI Service and Authentication Service into independent containers).
- **Multi-Tenant Database Scaling:** Migrate from a shared database schema to a multi-tenant model using schema-based isolation to ensure data separation for enterprise clients.
- **Enterprise Single Sign-On (SSO):** Implement federated authentication protocols (SAML 2.0, OIDC) to allow integration with enterprise identity providers.

- **Infrastructure as Code (IaC):** Define all cloud resources (Vercel, Render, Supabase, Cloudflare R2) using Terraform configuration scripts to support automated infrastructure replication.

16.6 AI Evolution

Table 16.1 details the planned roadmap for integrating advanced generative capabilities.

AI Feature	Timeline	Architectural Impact
AI Sprint Planning	6 Months	Computes velocity trends and generates automated sprint goals based on historical task completions.
AI Risk Assessment	9 Months	Evaluates task dependency chains and alerts project managers to potential timeline bottlenecks.
AI Productivity Insights	12 Months	Processes anonymized activity logs to generate collaboration recommendations.
Semantic Workspace Search	18 Months	Integrates pgvector to enable semantic natural-language search across task histories.

Table 16.1: Generative AI Evolution Roadmap

16.7 Cloud Infrastructure Evolution

As user counts grow, the cloud hosting topology must scale to ensure high availability:

- **Paid PaaS Hosting:** Upgrade from Render's free tier to paid Web Service tiers to eliminate container cold starts and access scaling groups.
- **Database Scaling:** Configure read-replica database nodes in Supabase and optimize PgBouncer connection limits to support higher read traffic.
- **Log Shipping:** Implement centralized log collectors (e.g., Grafana Loki) to collect container logs, supporting unified tracing using the `X-Request-ID` context parameter.

16.8 Technical Debt Reduction

Maintaining software quality requires a structured plan to address technical debt:

- **Schema De-duplication:** Consolidate duplicate Pydantic validation structures in the API tier, routing request types through shared schemas.
- **Repository Helper Refactoring:** Refactor database repository classes to share common SQL query logic, reducing code duplication.
- **Library Upgrades:** Update third-party Python packages and React client libraries to maintain compatibility and patch security vulnerabilities.

16.9 Architectural Milestones

Table 16.2 outlines key milestones, estimated timelines, and technical impacts for the platform's evolution.

Milestone	Priority	Timeline	Technical Impact
Paid Hosting Tier	High	1 Month	Eliminates container cold starts and enables scaling.
Asynchronous Queues	High	6 Months	Offloads long-running tasks from the main HTTP worker threads.
WebSockets Real-time	Medium	9 Months	Enables instant state updates across active user sessions.
SAML SSO Integration	Medium	12 Months	Adds enterprise-grade identity provider integration.
Terraform IaC Setup	Low	18 Months	Automates cloud infrastructure provisioning and replication.

Table 16.2: Architectural Evolution Milestones

16.10 Future Success Metrics

To measure the success of these architectural changes, the platform tracks key performance indicators:

- **System Performance:** Track P95 API latencies, aiming to keep core read operations below 80ms and write actions below 150ms.
- **Availability:** Monitor application uptime, targeting a 99.9% availability rate across all services.
- **Developer Velocity:** Measure the time required for a developer to set up a local environment and merge updates, aiming to reduce onboarding times.
- **AI Reliability:** Track LLM request errors and validation failures, targeting an output validation rate above 99.5%.

16.11 Chapter Summary

This chapter outlined the future roadmap and architectural evolution for Nexus PM. By defining clear short-term optimizations, medium-term enhancements (WebSockets and task queues), and long-term scaling strategies (microservices and IaC), the platform has a structured path from its current staging build to an enterprise-grade SaaS deployment. The final chapter will summarize the architectural decisions and conclusions of this document.

CHAPTER 17

Conclusion

17.1 Architectural Journey

The architectural journey of Nexus PM evolved from an initial concept of a collaborative workspace into a production-ready, multi-cloud SaaS platform. The development cycle began by defining core requirements (stateless operations, workspace isolation, and native generative AI features), which guided the selection of a decoupled React frontend and FastAPI backend.

Throughout the implementation, the architecture transitioned from a local-first system to a cloud-integrated platform. This transition involved:

- Integrating serverless cache registries (Upstash Redis) to manage rate-limiting.
- Offloading file attachments directly to S3-compatible cloud storage (Cloudflare R2) using pre-signed upload URLs.
- Leveraging managed databases (Supabase PostgreSQL) to ensure transactional safety.
- Deploying asynchronously coordinated LLM gateways (Google Gemini 2.5 Flash) to run task calculations.

This development process verified that the decoupled, modular monolith approach delivers high developer velocity, secure multi-tenancy, and low operational footprint on cost-constrained infrastructures.

17.2 Key Architectural Decisions

The architecture of Nexus PM is defined by six design decisions made during the system design:

- **Decoupled Frontend-Backend Model:** Separating the React 19 single-page application client from the FastAPI backend router enables independent component lifecycle management, caching, and scalability.
- **Modular Monolith backend:** Organizing backend logic into modular packages (Authentication, Projects, Tasks, Storage, AI) maintains simplicity while planning for future microservices partitioning if compute demands justify it.

- **Stateless API Gateways:** Sessions are validated statelessly using signed JSON Web Tokens (JWT), allowing backend containers to scale horizontally behind load balancers.
- **PgBouncer Connection Pooling:** Query requests connect securely using PgBouncer transaction-pooling layers, resolving database connection limits on the free tier.
- **Direct Object Storage Transfers:** Upload requests use temporary pre-signed R2 URLs, offloading file traffic from the API backend and eliminating egress bandwidth costs.
- **Asynchronous AI Pipelines:** Long-running LLM calls run asynchronously outside the primary request loop, preventing server thread blocking and maintaining fast response times.

17.3 Engineering Outcomes

The implementation of the Nexus PM architecture achieved several operational outcomes:

- **High Code Maintainability:** Enforcing strict separation of concerns (Repositories $\rightarrow$ Services $\rightarrow$ Routers) simplifies code updates, testing, and debugging.
- **Stateless Scalability:** Eliminating local session storage allows backend containers to scale dynamically based on request volumes.
- **Secure Multi-Tenancy:** Row-level access validations, secure HttpOnly cookie settings, and JWT verification parameters prevent data leakage across workspaces.
- **Developer Velocity (DX):** Packaging the application inside Docker containers, combined with automated linting, migration validations, and unit testing in CI pipelines, simplifies local onboarding and ensures deployment consistency.

17.4 Lessons Learned

Developing and deploying Nexus PM highlighted several key architectural trade-offs:

- **Compute Cost vs. System Availability:** Leveraging Render's free tier container hosting allowed for zero-cost prototyping but introduced container sleep cycles and cold start delays, which must be addressed before production launch.
- **Asynchronous Coordination Complexity:** Running AI generation jobs asynchronously kept the user interface responsive but required structured JSON validations to prevent malformed model outputs from causing application errors.
- **Managed Service Dependencies:** Relying on multiple managed cloud services (Supabase, Upstash, Cloudflare, Resend, Gemini) minimized database and cache administration overhead, but introduced external network failure risks that require robust error handling.

17.5 Overall Architectural Assessment

Nexus PM presents a secure, repeatable, and cost-efficient cloud-native architecture. The system successfully balances performance, security, and developer velocity by separating client interfaces from core business logic, leveraging serverless cloud resources, and natively embedding asynchronous AI pipelines.

The primary limitations of the staging deployment stem from free-tier infrastructure limits (cold starts, connection limits, and api rate quotas) rather than code design issues. Moving the platform to paid compute instances and configuring centralized APM dashboards will resolve these operational bottlenecks, ensuring a reliable, production-ready environment.

Table 17.1 outlines the current state of the architecture and our strategic direction for future scaling.

Architectural Area	Current Production State	Future Strategic Direction
Component Decoupling	Decoupled SPA client and modular FastAPI backend.	Service-level microservices refactoring.
Database Layer	Managed Supabase PostgreSQL with PgBouncer.	Read-replica database configurations.
Identity Management	Stateless JWT and secure HttpOnly cookie validation.	Federated SAML/OIDC SSO integrations.
AI Orchestration	Async Gemini API REST calls with JSON validation.	Local model hosting and semantic searches.
Deployment Host	Containerized free-tier Render PaaS deployments.	Auto-scaling groups on dedicated hosting instances.
Operations Tracking	Local console logging with request IDs.	Centralized APM monitoring dashboards.

Table 17.1: Nexus PM Architectural Status and Strategic Direction

17.6 Final Remarks

The Nexus PM Software Architecture Document establishes a reliable, scalable blueprint for a modern project management SaaS platform. By combining modern front-end libraries, high-performance back-end services, and contextual AI integrations with secure cloud infrastructure configurations, the system is fully prepared to transition from active development to a resilient, production-ready deployment.

CHAPTER A

Acronyms

This appendix provides a summary reference of the technical and architectural acronyms used throughout the Nexus PM Software Architecture Document.

Acronym	Full Expanded Definition
ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
APM	Application Performance Monitoring
ASGI	Asynchronous Server Gateway Interface
CDN	Content Delivery Network
CI/CD	Continuous Integration / Continuous Deployment
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
DX	Developer Experience
ERD	Entity-Relationship Diagram
HLD	High-Level Design
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLD	Low-Level Design
LLM	Large Language Model
NFR	Non-Functional Requirement
OAuth	Open Authorization
ORM	Object-Relational Mapping

Acronym	Full Expanded Definition
OTP	One-Time Password
PaaS	Platform-as-a-Service
RBAC	Role-Based Access Control
REST	Representational State Transfer
RLS	Row-Level Security
SaaS	Software-as-a-Service
SAD	Software Architecture Document
SMTP	Simple Mail Transfer Protocol
SPA	Single-Page Application
SSL	Secure Sockets Layer
TLS	Transport Layer Security
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
XSS	Cross-Site Scripting

CHAPTER B

Glossary

This glossary defines the technical terms, frameworks, concepts, and platforms referenced throughout this document to ensure clarity.

Artificial Intelligence (AI) Computational systems that simulate human cognitive processes, used in Nexus PM for task suggestion breakdown and context summaries.

Application Programming Interface (API) A structured set of protocols and rules that allows client applications to communicate with backend server resources.

Content Delivery Network (CDN) A globally distributed network of proxy servers that cache static assets close to users, utilized by Vercel for the Nexus PM React client.

Continuous Integration / Continuous Deployment (CI/CD) Automated software workflows that format, lint, scan, test, and package code modifications into staging or production builds on every repository push.

Cloud-Native Architectures designed specifically to execute within distributed cloud compute platforms, leveraging managed APIs and serverless services.

Container A lightweight, standalone, and executable package of software that contains code, runtimes, system tools, libraries, and configurations.

Create, Read, Update, Delete (CRUD) The four basic operations of persistent storage, representing the core capabilities of project and task tracking systems.

Docker The virtualization software used to package the Nexus PM FastAPI backend and React frontend into consistent, portable container environments.

Entity-Relationship Diagram (ERD) A database visualization mapping relational tables, column attributes, primary key linkages, and foreign key boundaries.

FastAPI A high-performance, async-first Python ASGI framework used to build RESTful API endpoints with automatic OpenAPI schema documentation.

Gemini Google's multimodal generative AI model, integrated via the Google GenAI SDK to handle project and task text processing.

Hypertext Transfer Protocol (HTTP) The foundational stateless client-server communication protocol of the Web, secured via TLS/SSL (HTTPS) in production.

JSON Web Token (JWT) An open standard (RFC 7519) that defines a compact, self-contained way to securely transmit identity information as a JSON object.

JavaScript Object Notation (JSON) A lightweight data-interchange format that is easy for humans to read and write and easy for machines to parse.

Kanban An agile project management visualization framework displaying active task items as cards across sequential columns (Todo, In Progress, Review, Done).

Large Language Model (LLM) Deep learning neural networks trained on massive text datasets, capable of understanding and synthesizing human language.

Open Authorization (OAuth) An open standard framework for token-based authorization on the Internet, used via Google OAuth to authenticate users.

Object-Relational Mapping (ORM) A programming technique that maps database tables to classes in code, enabling developers to query databases using object-oriented code.

PgBouncer A lightweight connection pooler for PostgreSQL, running on Supabase to route transient API queries to a fixed pool of database connections.

PostgreSQL A highly stable, ACID-compliant open-source relational database management system, serving as the main data store for Nexus PM.

Role-Based Access Control (RBAC) An authorization paradigm where access permissions are mapped to roles, and roles are assigned to users within project scopes.

React An open-source, component-based frontend JavaScript UI library managed by Meta, used to build interactive single-page user interfaces.

Redis An open-source, in-memory key-value data structure store used for low-latency session caches, dashboard analytics caches, and rate-limiting counters.

Representational State Transfer (REST) An architectural style for designing networked applications, using HTTP verbs (GET, POST, PUT, DELETE) to manage resource states.

Software-as-a-Service (SaaS) A software licensing and delivery model in which software is centrally hosted and licensed on a subscription basis.

SQLAlchemy The SQL toolkit and Object-Relational Mapper for Python, providing database connectivity and query generation for Nexus PM.

Supabase A developer-friendly Backend-as-a-Service (BaaS) provider that hosts the managed PostgreSQL instance, connection pools, and automatic backups.

Tailwind CSS A utility-first CSS framework used to compose clean, responsive user interfaces directly within React HTML/JS structures.

Vercel The cloud hosting platform used to distribute the static React frontend client using its global Edge CDN nodes.

Vite A modern, high-speed frontend build tool that manages dev server hot-module replacement and bundles client assets using Rollup.

CHAPTER C

Technology Selection Rationale

This appendix documents the trade-off evaluations and decisions that guided the technology selections for the Nexus PM platform. It outlines the alternative technologies considered and details the specific reasons why the chosen stack was selected to satisfy performance, budget, and operational objectives.

Technology	Selected Over	Architectural & Operational Rationale
FastAPI	Flask / Django	Native asynchronous support, auto-generated OpenAPI documentation, and built-in Pydantic data validation constraints.
React 19	Angular / Vue	Flexible component-based rendering model, vast developer ecosystem, and compatibility with Vite and React Query.
Supabase	Self-hosted PostgreSQL	Fully managed PostgreSQL backend, automatic system backups, and zero maintenance overhead within free-tier limits.
Cloudflare R2	AWS S3	Zero egress data transfer bandwidth billing, S3-compatible API, and predictable hosting costs.
Upstash Redis	Self-hosted Redis	Serverless pricing structure, automated scale-to-zero capabilities, and low-latency cache queries without node provisioning.
Resend API	AWS SES	Developer-friendly integration APIs, straightforward domain validation settings, and native HTML template parsing.
Gemini 2.5 Flash	Other LLMs (GPT-4o)	Generous free-tier API quotas, low request latency, large context window, and native support for strict JSON schema output requirements.

Table C.1: Nexus PM Core Technology Selection Trade-offs

Bibliography

[1] Cloudflare r2 documentation, 2026.

[2] Docker documentation, 2026.

[3] Fastapi documentation, 2026.

[4] Google gemini api documentation, 2026.

[5] Postgresql documentation, 2026.

[6] React official documentation, 2026.

[7] Supabase documentation, 2026.

[8] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley Professional, 2021.

[9] D. Hardt. The oauth 2.0 authorization framework. RFC 6749, IETF, 2012.

[10] M. Jones, J. Bradley, and N. Sakimura. Json web token (jwt). RFC 7519, IETF, 2015.

[11] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

[12] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.

[13] Leonard Richardson and Mike Amundsen. *RESTful Web APIs*. O'Reilly Media, 2013.